MÁSTER EN ANÁLISIS DE MALWARE Y REVERSING

MÓDULO 8. REVERSING SISTEMAS OPERATIVOS (MÓVILES)

MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*



Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Contenido

1. INTRODUCCIÓN 4
2. ENFOQUE DEL MÓDULO DE ANÁLISIS EN DISPOSITIVOS MÓVILES 4
3. METODOLOGÍA 5
3.1. OBJETIVOS 6
3.2. OBTENCION DE LA APLICACIÓN 6
3.2.1. ANDROID 6
3.2.2. IOS 7
3.3. COMPRENSIÓN DE LA APLICACIÓN 7
3.4. ANÁLISIS ESTÁTICO AVANZADO DEL BINARIO 7
3.5. ANÁLISIS DINÁMICO AVANZADO 8
3.6. INFORME RESULTADO 8
4. VULNERABILIDADES EN DISPOSITIVOS MÓVILES 9
Imagen 1 OWASP Mobile Top 10 Risk 9
5. PLATAFORMAS 10
5.1. ANDROID 10
5.1.1. LENGUAJES 11
5.1.2. COMPONENETES ANDROID 13
1. Iniciar una actividad 16
2. Iniciar un servicio 16
3. Transmitir una emisión 16
5.1.3. ESTRUCTURA ANDROID 17
5.2. IOS 19
5.2.1. LENGUAJE IOS 20
5.2.2. COMPONENETES IOS 20
5.2.3. ESTRUCTURA IOS 22
5.3. WINDOWS PHONE 26
6. ENTORNO DE ANÁLISIS Y HERRAMIENTAS 26
7. PUNTOS DE ENTRADA ANDROID 29
7.1. ARCHIVO MANIFIEST 29
7.2 PERMISOS INNECESARIOS 29
7.3 DATOS SENSIBLES EN LAS CADENAS DE TEXTO 29

7.4 SEGURIDAD DE TRANSPORTE 30
7.5 ACTIVIDAD DEL LANZADOR 30
7.6 SERVICIOS 31
7.7 BROADCAST RECEIVERS 32
7.8 COMPONENTES EXPORTADOS (SERVICIOS Y ACTIVIDADES) 32
7.9 SUBCLASE DE APLICACIÓN 32
8. PUNTOS DE ENTRADA IOS 32
8.1. ARCHIVO INFO.PLIST 33
8.2. PERMISOS INNECESARIOS 33
8.3. DATOS SENSIBLES EN LAS CADENAS DE TEXTO 33
8.4. SEGURIDAD DE TRANSPORTE 33
8.5. ANÁLISIS DEL BINARIO DE LA APLICACIÓN 34
8.6. GENERADOR DE NÚMEROS ALEATORIOS INSEGURO 34
8.7. USO DE ALGORITMOS HASH DÉBILES 34
8.8. USO DE BANNED/DEPRECATED APIS 34
8.9. BINARIO DE APLICACIÓN SIN CIFRAR 35
8.10. FLAG DE COMPILACIÓN 35
9. COMPRENSIÓN DE LA APLICACIÓN 35
9.1. COMPRENSIÓN DE LA APLICACIÓN EN LA PRÁCTICA 36
10. ANÁLISIS ESTÁTICO 38
10.2. DESEMPAQUETADO DE LA APLICACIÓN 38
10.3. ANÁLISIS ESTÁTICO EN ANDROID 39
10.4. ANÁLISIS ESTÁTICO EN LA PRÁCTICA 40
10.5. ANÁLISIS ESTÁTICO EN IOS 58
11. ANÁLISIS DINÁMICO 69
11.2. EJECUCIÓN DE LA APLICACIÓN 69
11.3. ANÁLISIS DINÁMICO EN ANDROID 70
11.5. ANÁLISIS DINÁMICO EN LA PRÁCTICA 70
12. EJERCICIOS PRÁCTICOS 77
13. REFERENCIAS 78

1. INTRODUCCIÓN

A lo largo de este módulo se estudiará todo lo necesario para poder llevar a cabo el análisis de aplicaciones móviles mediante reversing, tanto Android como iOS. Existen diferentes razones prácticas por las que querríamos analizar una aplicación, pero básicamente nos centraremos en dos: la primera es para ver si la aplicación es vulnerable a algún tipo de ataque y la otra es para ver si se trata de malware.

En este módulo pondremos foco en la infraestructura, herramientas, metodología y búsqueda de vulnerabilidades, dado que en el siguiente módulo del curso existe un capítulo específicamente dedicado a análisis de malware en dispositivos móviles.

Se aprenderá a preparar el entorno necesario para realizar el análisis de aplicaciones iOS y Android. Para ello vamos a utilizar una serie de herramientas, todas gratuitas y disponibles al alcance de cualquier usuario, que nos van a ayudar a extraer la información necesaria de las aplicaciones que queremos analizar. Vamos a obviar la instalación de las máquinas virtuales, puesto que son instalaciones de sistemas operativos en sí, y nos centraremos en la instalación y configuración de las herramientas que puedan suponer una mayor complejidad. Si bien algunas se entregarán preinstaladas, es preferible realizar la instalación uno mismo para entender exactamente el objetivo y funcionamiento de la herramienta en cuestión. Ha de tenerse en cuenta que existen multitud de herramientas, se mostrará el despliegue y uso de varias de ellas y se añadirá un capítulo final con diferentes herramientas y frameworks de interés.

Es importante señalar que el análisis en aplicaciones Ios, en una parte requiere de dispositivo rooteado, motivo por el cual no se abordarán las áreas de análisis que impliquen la compra y modificación de dispositivos.

2. ENFOQUE DEL MÓDULO DE ANÁLISIS EN DISPOSITIVOS MÓVILES

Una de las principales claves de la ingeniería inversa, independientemente de la plataforma, es averiguar el objetivo final y dónde comenzar el análisis. Como ya se ha explicado en la introducción, **pondremos foco en este capítulo en la construcción de la infraestructura para el análisis de aplicaciones móviles y concretamente en el análisis de vulnerabilidades en aplicaciones móviles**.

Si bien una de las partes del proceso que más información aportarán será el análisis de código, no se profundizará dado que se ha impartido un módulo específico de análisis de código a lo largo del master, el cual será aplicable en esta sección. Si se verá en detalle con ejemplos prácticos como

poder realizar un análisis completo comprendiendo las estructuras de cada plataforma, las técnicas de procesamiento y los análisis necesarios.
Ha de tenerse en cuenta, que en muchas ocasiones una app móvil no es más que un frontal de una aplicación web. En este capítulo no se analizarán las vulnerabilidades del lado del servidor, pondremos foco en entender las funcionalidades y posibles vulnerabilidades que presente el aplicativo móvil.
Para finalizar, es muy interesante si no se dispone de conocimiento de aplicaciones móviles, que se trate de crear una aplicación básica que nos ayude a entender la morfología y componentes de este tipo de plataformas. Existen múltiples recursos gratuitos que nos permitirán construir una de forma rápida y sencilla. En esta referencia podréis encontrar todo lo necesario: **Enlace**

3. METODOLOGÍA

Seguiremos una serie de pasos estructurados para realizar el análisis de aplicaciones móviles.



Imagen 0 **Esquema metodología de análisis**

3.1. OBJETIVOS

El primer paso será determinar el Objetivo que perseguimos con nuestro análisis

Se debe realizar un análisis del binario final para determinar entre otros el código fuente, las bibliotecas, los algoritmos y los recursos incorporados en la aplicación.

Se puede usar la ingeniería inversa para lograr cualquiera de los siguientes objetivos:

- Revelar información sobre los servidores Backend
- Revelar constantes y cifrados criptográficos
- Robar propiedad intelectual
- Realizar ataques contra los Backend
- Obtener la inteligencia necesaria para realizar la subsiguiente modificación del código.

Generalmente, la mayoría de las aplicaciones son susceptibles a la ingeniería inversa debido a la naturaleza inherente del código. La mayoría de los lenguajes usados para escribir aplicaciones hoy en día son ricos en metadatos que ayudan enormemente a un programador a depurar la aplicación. Esta misma capacidad también ayuda enormemente a un atacante a entender cómo funciona la aplicación. Se dice que una aplicación es susceptible de ingeniería inversa si un atacante puede hacer cualquiera de las siguientes cosas:

- Entender claramente el contenido de la tabla de cadenas de un binario
- Realizar con precisión el análisis multifuncional
- Derivar una recreación razonablemente precisa del código fuente del binario.

3.2. OBTENCION DE LA APLICACIÓN

Normalmente se obtendrá la aplicación objetivo de diversas formas, desde la extracción de un dispositivo hasta la descarga de la aplicación de una tienda de aplicaciones. Será analizada dentro de su propio entorno local (Siempre y cuando sea posible) utilizando un conjunto de herramientas.

3.2.1. ANDROID

Si el archivo APK está protegido utilizando cualquier software comercial de protección, entonces no podremos ver el código de forma legible, o si el código está ofuscado entonces también será difícil leer el código después de la ingeniería inversa.
Hay muchos ofuscadores gratuitos y de grado comercial en el mercado. A la inversa, hay muchos desofuscadores diferentes en el mercado.

Un buen ofuscador tendrá las siguientes habilidades:

- Reducir los métodos / segmentos de código a ofuscar;
- Afinar el grado de ofuscación para equilibrar el impacto del rendimiento; Ofuscar tablas de cadenas, así como métodos

3.2.2. IOS

Una aplicación de la App Store de Apple está cifrada con un esquema criptográfico respaldado por hardware. Más concretamente, una sección ejecutable del binario O-Mach dentro del paquete IPA está cifrada, y la clave de descifrado es accesible sólo en un dispositivo concreto a nivel de hardware (Secure Enclave). En todo caso sí es posible descifrar una aplicación descargada de la App Store de Apple para realizar un análisis estático. **El problema es que ES NECARIO UN DISPOSITIVO ROOTEADO,** ya que lo que se hace es ejecutar la aplicación y dumpearla de memoria. Hay 2 enfoques para volcar la región ejecutable descifrada de la memoria al sistema de archivos. Todos ellos requieren privilegios de superusuario, ya sea para rastrear un proceso, o para inyectar una biblioteca dinámica.
En nuestro caso siempre partiremos del fichero. ipa para su análisis.

Seguidamente se realizará análisis estático, es decir, analizar el código de la aplicación sin ejecutarla.

3.3. COMPRENSIÓN DE LA APLICACIÓN

Abriremos la aplicación en un entorno de emulación (Siempre que sea posible por el entorno), para analizarla funcionalmente. Si bien se trata análisis dinámico, suele ayudar de forma previa al análisis estático. Nos apoyaremos también en los primeros datos que observemos estáticamente. Los objetivos de esta fase son:

- **Entender funcionamiento de la app a nivel usuario**
- **Entender su lógica, usuarios y privilegios**
- **Cuál es el flujo de la comunicación entre objetos**
- **Cuando se produce una comunicación con el servidor**
- **Atención especial a eventos de la aplicación**
- **Infraestructura de la aplicación**

3.4. ANÁLISIS ESTÁTICO AVANZADO DEL BINARIO

Analizaremos el código fuente del binario, sus string y cualquier detalle relevante tras el desempacado inicial. Las principales tareas de esta fase son:

- **Información genérica del binario**
- **Información de sub-binarios y arquitecturas**
- **Volcado de clases**
- **Análisis de símbolos**
- **Análisis de dependencias**
- **Análisis de strings**
- **Comprobación de cifrado de la aplicación**
- **Comprobación de activación de ASLR (PIE)**
- **Desensamblado**
- **Depuración**

3.5. ANÁLISIS DINÁMICO AVANZADO

Análisis en tiempo de ejecución. Es decir, aprovecharemos información que sólo se puede obtener en tiempo de ejecución.

- **Aprovechar información obtenida de símbolos**
- **Aprovechar información obtenida de strings**
- **Realizar volcado de clases**
- **Analizar métodos, funciones, argumentos…**
- **Ejecutar la aplicación en dispositivo Virtual en caso de ser posible**
- **Hacer "hook" (engancharnos) al proceso**
- **Invocar métodos en tiempo de ejecución**

El proceso será diferente en función del tipo de plataforma sobre la que se ha construido la aplicación, ya que las técnicas y herramientas son diferentes en función de la tecnología.

Se realizará análisis Automátizado mediante herramientas. Estas herramientas nos ayudarán a realizar tareas rutinarias y tediosas y nos presentarán la información de forma visual. Nos aportarán la posibilidad de detectar vulnerabilidades en la aplicación sin realizar tareas complejas de análisis. Obviamente será más rápido y sencillo, pero sus capacidades de detección son limitadas, por lo que la interpretación del código fuente y otras tareas manuales del analista serán necesarias.

3.6. INFORME RESULTADO

Como siempre que se realiza el análisis de una aplicación en cualquier ámbito, deberán presentarse los resultados de forma ordenada y aportando evidencias. Se identificarán las vulnerabilidades detectadas y se asignará un riesgo.

4. VULNERABILIDADES EN DISPOSITIVOS MÓVILES

No es el objetivo de este curso aprender sobre cómo hacer pentesting en dispositivos móviles, sino más bien entender los mecanismos y herramientas para realizar ingeniería inversa sobre dispositivos móviles. Pero para tratar de hacer el capítulo más interesante trataremos de darle un propósito concreto, el cual será la búsqueda de vulnerabilidades mediante reversing en este tipo de aplicaciones.

Tomaremos como referencia base las siguientes Metodologías para análisis de aplicaciones móviles. El objetivo será tener un marco que nos ayude durante el proceso.

- **OWASP Mobile TOP 10 Risks** - OWASP Mobile Security Project Android
- **OASAM** - Open Android Security Assessment Methodology.

Por este motivo comenzaremos detallando a continuación los 10 riesgos mas habituales en dispositivos móviles según OWASP



Imagen 1 OWASP Mobile Top 10 Risk

OASAM, es el acrónimo de Open Android Security Assessment Methodology, que pretende ser una metodologia de análisis de seguridad de aplicaciones para la plataforma Android. Cuenta con controles de seguridad que se estructuran en las siguientes secciones:

- **OASAM-INFO** - Information Gathering: Obtención de información y defición de superficie de ataque.

- **OASAM-CONF** - Configuration and Deploy Management: Análisis de la configuración e implantación.
- **OASAM-AUTH** – Authentication: Análisis de la autenticación.
- **OASAM-CRYPT** – Cryptography: Análisis del uso de criptografía.
- **OASAM-LEAK** - Information Leak: Análisis de fugas de información sensible.
- **OASAM-DV** - Data Validation: Análisis de gestión de la entrada de usuario.
- **OASAM-IS** - Intent Spoofing: Análisis de la gestión en la recepción de Intents.
- **OASAM-UIR** - Unauthorized Intent Receipt: Análisis de la resolución de intents.
- **OASAM-BL** - Business Logic: Análisis de la lógica de negocio la aplicación.

El conocer en detalle los tipos de vulnerabilidades nos ayudará a saber interpretar el resultado de las herramientas automáticas, y obviamente son el foco del análisis.

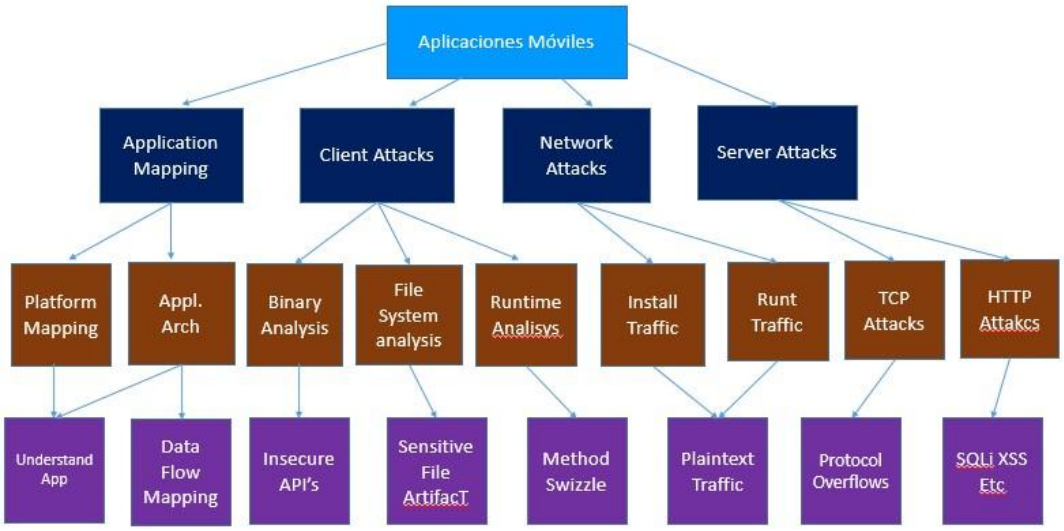


Imagen 2 **Esquema metodología de análisis**

5. PLATAFORMAS

5.1. ANDROID

Las aplicaciones diseñadas para Android tienen la extensión ".apk". Este formato no es más que una variante del formato JAR de Java y se usa para instalar y distribuir aplicaciones Android. Por normal general las aplicaciones para Android están escritas en lenguaje Java, aunque no es posible utilizar una máquina virtual Java como OpenJDK o JVM, sino que se ejecutan en una máquina virtual "especial" diseñada para dispositivos con poca potencia o embebidos, llamada Dalvik.

Con el paso del tiempo y con la inquietud de muchos investigadores de seguridad, se han conseguido desarrollar una serie de "modificaciones" sobre los diferentes sistemas operativos con

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

el fin de que un usuario final pueda tener acceso a partes del dispositivo y/o funciones extra. En el caso de Android, a este proceso se le conoce como Rooting. Ésta técnica va ligada, en muchos casos, al tipo de dispositivo sobre el que corra el sistema operativo (teléfonos móviles, tablets, dispositivos embebidos...) y ofrece al usuario final la opción de poder ejecutar comandos y aplicar una serie de modificaciones sobre el sistema operativo que no sería posible realizar de otro modo, como puede ser obtener acceso como usuario administrador al dispositivo.

5.1.1. LENGUAJES

Desde el punto de vista de los lenguajes implicados en la creación de una aplicación y su proceso de reverseado, los formatos de código que encontraremos a lo largo del proceso son:

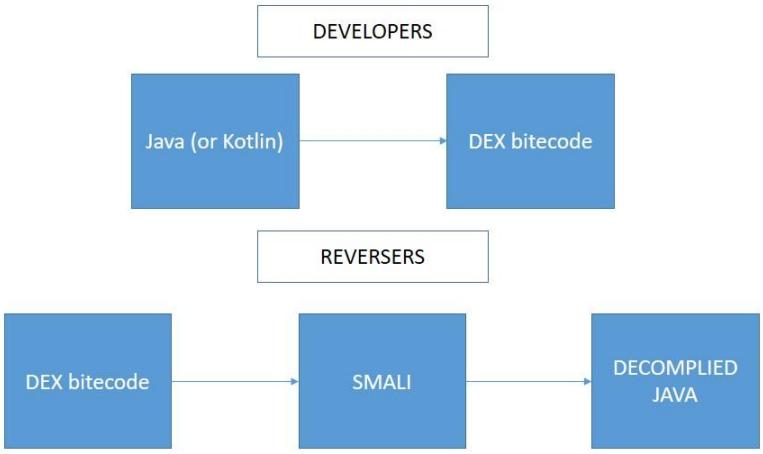


Imagen 3 **Lenguajes implicados**

Smali es la versión legible para humanos del bytecode Dalvik. Técnicamente, Smali y baksmali son el nombre de las herramientas (ensamblador y desensamblador, respectivamente), pero en Android, a menudo utilizamos el término "Smali" para referirnos a las instrucciones. Si has hecho ingeniería inversa o arquitectura de ordenadores sobre código C/C++ compilado. SMALI es como el lenguaje ensamblador: entre el código fuente de nivel superior y el bytecode.

Para el siguiente código Java Hola Mundo:

```
public static void printHelloWorld() {

        System.out.println("Hello World")

}
```

El código Smali sería:

```
.method public static printHelloWorld()V

            .registers 2        sget-object v0, Ljava/lang/System;->out:Ljava/io/PrintStream;

const-string v1, "Hello World"  invoke-virtual {v0,v1}, Ljava/io/PrintStream;-

>println(Ljava/lang/String;)V

                return-void

.end method
```

Información sobre el set de instrucciones Smali: **Enlace**

Generalmente no será necesario trabajar en Smali. La mayoría de las aplicaciones pueden ser elevadas a un nivel superior, descompilando Java.

Pero en ocasiones las herramientas no funcionaran correctamente por diversos motivos, y será muy útil trabajar con la salida Smali de forma paralela, comparando línea a línea para poder entender el código java.

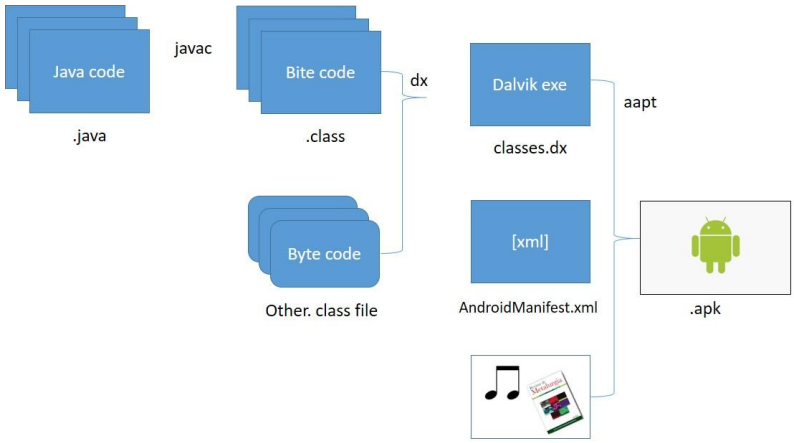


Imagen 4 **Esquema Android**

5.1.2. COMPONENETES ANDROID

A la hora de diseñar cualquier tipo de aplicación estamos acostumbrados a manejar conceptos como eventos, servicios, controles… Pues bien, Android también tiene sus propios componentes, utilizados para la interacción entre los mismos componentes y entre los componentes y el usuario. Los componentes en los que se divide una aplicación Android son:

- **Activities:** En resumen, puede decirse que son componentes utilizados para pintar la interfaz del usuario. Una Activity o *actividad* es el punto de entrada de interacción con el usuario. Representa una pantalla individual con una interfaz de usuario. Por ejemplo, una aplicación de correo electrónico tiene una actividad que muestra una lista de los correos electrónicos nuevos, otra actividad para redactar el correo electrónico y otra actividad para leerlo. Si bien las actividades funcionan juntas para ofrecer una experiencia de usuario uniforme en la aplicación de correo electrónico, cada una es independiente de las demás. De esta manera, una aplicación diferente puede iniciar cualquiera de estas actividades (si la aplicación de correo electrónico lo permite). Por ejemplo, para que el usuario comparta una imagen, una aplicación de cámara puede iniciar la actividad correspondiente en la aplicación de correo electrónico que redacta el nuevo mensaje. Una actividad posibilita las siguientes interacciones clave entre el sistema y la aplicación:
    - Realizar un seguimiento de lo que realmente le interesa al usuario (lo que está en pantalla) para garantizar que el sistema siga ejecutando el proceso que aloja la actividad.
    - Saber que los procesos usados con anterioridad contienen elementos a los que el usuario puede regresar (actividades detenidas) y, en consecuencia, priorizar más esos procesos que otros.
    - Ayudar a la aplicación a controlar la finalización de su proceso para que el usuario pueda regresar a las actividades con el estado anterior restaurado.
    - Permitir que las aplicaciones implementen flujos de usuarios entre sí y que el sistema los coordine (el ejemplo más común es compartir).

- **Services:** En resumen, podemos decir que son servicios ejecutados en segundo plano y suelen realizar procesos de larga duración, procesos remotos, etc. Un *servicio* es un punto de entrada general que permite mantener la ejecución de una aplicación en segundo plano por diversos motivos. Es un componente que se ejecuta en segundo plano para realizar operaciones de ejecución prolongada o para realizar tareas de procesos remotos. Un servicio no proporciona una interfaz de usuario. Por ejemplo, un servicio podría reproducir música en segundo plano mientras el usuario se encuentra en otra aplicación, o podría capturar datos en la red sin bloquear la interacción del usuario con una actividad. Otro componente, como una actividad, puede iniciar el servicio y permitir que se ejecute o

enlazarse a él para interactuar. De hecho, existen dos semánticas muy diferentes que los servicios usan para indicarle al sistema cómo administrar una aplicación:

Los servicios iniciados le indican al sistema que los siga ejecutando hasta que finalicen su trabajo. Dos ejemplos serían la sincronización de datos en segundo plano o la reproducción de música después de que el usuario sale de la aplicación. La sincronización de datos en segundo plano o la reproducción de música también son dos tipos de servicios iniciados muy diferentes que modifican la manera en que el sistema los administra:

- La reproducción de música es algo de lo que el usuario es consciente, por lo que la aplicación se lo comunica al sistema indicándole que quiere ejecutarse en segundo plano y le envía una notificación al respecto al usuario. En este caso, el sistema sabe que debe hacer todo lo posible para mantener el proceso de ese servicio en funcionamiento porque el usuario no estará satisfecho si se interrumpe.
- Un servicio habitual en segundo plano no es algo de lo que el usuario sea consciente, por lo que el sistema tiene más libertad para administrarlo. Puede permitir que se interrumpa (y reiniciarlo posteriormente) si necesita memoria RAM en procesos que son más urgentes para el usuario.

Los servicios enlazados se ejecutan porque otra aplicación (o el sistema) indicó que quiere usarlos. Básicamente, lo que sucede es que un servicio le brinda una API a otro proceso. Por lo tanto, el sistema sabe que hay una dependencia entre estos procesos. En consecuencia, si el proceso A está enlazado a un servicio en el proceso B, sabe que debe mantener funcionando el proceso B (y el servicio correspondiente) para el proceso A. Además, si el proceso A es de interés para el usuario, también sabe que debe tratar el proceso B teniendo esto en cuenta. Gracias a su flexibilidad (o a pesar de ella), los servicios se han convertido en un componente muy útil para todos los tipos de conceptos generales del sistema. Los fondos de pantalla animados, los receptores de notificaciones, los protectores de pantalla, los métodos de entrada, los servicios de accesibilidad y muchas otras funciones básicas del sistema se compilan como servicios que las aplicaciones implementan y que el sistema vincula cuando deben ejecutarse.

- **Content Providers:** Podemos decir de forma sencilla que proveen el contenido. Administra y comparte contenido entre aplicaciones.

Un *proveedor de contenido* administra un conjunto compartido de datos de la aplicación que puedes almacenar en el sistema de archivos, en una base de datos SQLite, en la Web o en cualquier otra ubicación de almacenamiento persistente a la que tenga acceso tu aplicación. A través del proveedor de contenido, otras aplicaciones pueden consultar o modificar los datos si el proveedor de contenido lo permite. Por ejemplo, el sistema Android proporciona un proveedor de contenido que administra la información de contacto del usuario. De esta manera, cualquier aplicación con los permisos correspondientes puede consultar el proveedor de contenido (como ContactsContract.Data) para la lectura y la escritura de información sobre una persona específica. En ocasiones, se interpreta que el proveedor de contenido es una abstracción en una base de

datos, porque tiene un gran volumen de API y compatibilidad incorporado. Sin embargo, su finalidad principal es diferente desde una perspectiva de diseño del sistema. Para el sistema, un proveedor de contenido es un punto de entrada a una aplicación para publicar elementos de datos con nombre y se identifica mediante un esquema de URI. Así, una aplicación puede decidir cómo quiere asignar los datos que contiene a un espacio de nombres de URI y entregar esos URI a otras entidades que, a su vez, pueden usarlos para acceder a los datos. Gracias a esto, el sistema puede realizar algunas tareas específicas cuando administra una aplicación:

- Asignar un URI no exige que la aplicación permanezca ejecutándose, por lo que los URI pueden persistir incluso después de que se cierran las aplicaciones a las que pertenecen. El sistema solo necesita asegurarse de que la aplicación de un URI siga ejecutándose si debe recuperar los datos de la aplicación desde el URI en cuestión.
- Estos URI también ofrecen un modelo de seguridad importante y detallado. Por ejemplo, una aplicación puede colocar el URI de una imagen que tiene en el portapapeles, pero bloquear al proveedor de contenido para que otras aplicaciones no puedan acceder a él libremente. Si otra aplicación intenta acceder a ese URI en el portapapeles, el sistema puede concederle acceso usando un *permiso de URI* temporal para que solo pueda acceder a los datos mediante ese URI, pero nada más.

Los proveedores de contenido también son útiles para leer y escribir datos privados de tu aplicación y que no se comparten.

- **Broadcast Receivers:** Simplificando podemos decir que permiten registrar eventos que se produzcan en el sistema operativo. Un Broadcast Receivers o receptor *de emisión,* es un componente que posibilita que el sistema entregue eventos a la aplicación fuera de un flujo de usuarios habitual, lo que permite que la aplicación responda a los anuncios de emisión de todo el sistema. Dado que los receptores de emisión son otro punto bien definido de entrada a la aplicación, el sistema puede entregar emisiones incluso a las aplicaciones que no estén en ejecución. Por ejemplo, una aplicación puede programar una alarma para publicar una notificación sobre un evento futuro destinada al usuario. Al entregar dicha alarma al receptor de emisión de la aplicación, no hace falta que dicha aplicación siga ejecutándose hasta que se active la alarma. Muchas emisiones provienen del sistema (por ejemplo, las que anuncian que se apagó la pantalla, que el nivel de carga de la batería es bajo o que se capturó una imagen). Las aplicaciones también pueden iniciar emisiones, por ejemplo, para avisarles a las demás aplicaciones que se descargaron datos al dispositivo y que están disponibles para el uso. Si bien los receptores de emisión no exhiben una interfaz de usuario, pueden crear una notificación de la barra de estado para alertar al usuario cuando se produzca un evento de emisión. Sin embargo, por lo general, un receptor de emisión es simplemente una *puerta de enlace* a otros componentes y está

destinado a realizar una cantidad mínima de trabajo. Por ejemplo, podría programar un servicio JobService para que realice algunas tareas en función del evento con JobScheduler.

- **Views:** Son los componentes de la interfaz del usuario (botones, cuadros de texto...)

- **Intents:** Son mensajes de notificación entre servicios y actividades.

Se trata de un objeto de mensajería que se puede usar para solicitar una acción de otro componente de una app. Si bien las intents facilitan la comunicación entre componentes de varias formas, existen tres casos de uso principales:

1. Iniciar una actividad

- Una Activity representa una única pantalla en una aplicación. Puedes iniciar una nueva instancia de una Activity pasando una Intent a startActivity(). La Intent describe la actividad que se debe iniciar y contiene los datos necesarios para ello.

- Si deseas recibir un resultado de la actividad cuando finalice, llama a startActivityForResult(). La actividad recibe el resultado como un objeto Intent separado en la devolución de llamada de onActivityResult() de la actividad. Para obtener más información, consulta la guía Actividades.

2. Iniciar un servicio

- Un Service es un componente que realiza operaciones en segundo plano sin una interfaz de usuario. Con Android 5.0 (nivel de API 21) y versiones posteriores, puedes iniciar un servicio con JobScheduler. Para obtener más información acerca de JobScheduler, consulta su API-reference documentation.

- En las versiones anteriores a Android 5.0 (nivel de API 21), puedes iniciar un servicio usando métodos de la clase Service. Puedes iniciar un servicio para realizar una operación única (como descargar un archivo) pasando una Intent a startService(). La Intent describe el servicio que se debe iniciar y contiene los datos necesarios para ello.

- Si el servicio está diseñado con una interfaz cliente-servidor, puedes establecer un enlace con el servicio de otro componente pasando una Intent a bindService(). Para obtener más información, consulta la guía Servicios.

3. Transmitir una emisión

- Una emisión es un aviso que cualquier aplicación puede recibir. El sistema transmite varias emisiones de eventos, como cuando se inicia el sistema o comienza a cargarse el dispositivo. Puedes transmitir una emisión a otras apps pasando una Intent a sendBroadcast() o sendOrderedBroadcast().

- **Manifest:** Este componente hace referencia al fichero AndroidManifest.xml. Está situado en la raíz de la aplicación. Es un fichero de configuración que proporciona información importante sobre la aplicación al sistema Android: declara los permisos que tendrá la aplicación, declara los permisos que otras aplicaciones han de tener para poder comunicarse con la nuestra, describe los componentes de la aplicación... Cada vez que se realiza y se publica una nueva versión de la aplicación, ésta ha de quedar reflejada en este fichero (parecido a un control de versiones).

Todos los proyectos de apps deben tener un archivo `AndroidManifest.xml` (con ese mismo nombre) en la raíz de la fuente del proyecto. El archivo de manifiesto describe información esencial de tu aplicación para las herramientas de creación de Android, el sistema operativo Android y Google Play.

Entre muchas otras cosas, el archivo de manifiesto debe declarar lo siguiente:

- El nombre del paquete de la aplicación, que normalmente coincide con el espacio de nombres de tu código. Las herramientas de compilación de Android usan esto para determinar la ubicación de las entidades de código cuando se compila el proyecto. Al empaquetar la aplicación, las herramientas de compilación sustituyen este valor por el ID de aplicación de los archivos de compilación de Gradle, que se utiliza como identificador único de la aplicación en el sistema y en Google Play.
- Los componentes de la aplicación, que incluyen todas las actividades, servicios, receptores de emisiones y proveedores de contenido. Cada componente debe definir propiedades básicas, como el nombre de su clase Kotlin o Java. También puede declarar capacidades, como las configuraciones de dispositivos que puede manejar, además de filtros de intents que describen cómo se puede iniciar el componente.
- Los permisos que necesita la aplicación para acceder a las partes protegidas del sistema o a otras aplicaciones. También declara cualquier permiso que otras aplicaciones deben tener si quieren acceder al contenido de esta aplicación.
- Las funciones de hardware y software que requiere la aplicación afectan a los dispositivos que pueden instalar la aplicación desde Google Play.

5.1.3. ESTRUCTURA ANDROID

El sistema operativo Android requiere de un árbol de directorios/ficheros especifico, donde cada carpeta ha de guardar una serie de elementos o recursos de la aplicación en concreto. Por norma general, el conjunto de carpetas y ficheros que nos vamos a encontrar a la hora de reversear una aplicación Android serían los siguientes:



Imagen 5 **Estructura apk**

Las aplicaciones android, se distribuyen en formato apk, que no es más un contenedor zip que está basado en un contenedor de archivos java, esta estructura una vez desensamblada nos muestra los siguientes archivos contenidos:

- **AndroidManifest.xml:** Ya hemos visto anteriormente su cometido.
- **classes.dex**: Este fichero contiene código ejecutable Android preparado para la máquina virtual Dalvik. Es un código similar a los "byte code" utilizados en máquinas virtuales de Java, en los ficheros .class. Obviamente este será el fichero más interesante para realizar reversing del código.

- **resources.arsc**: Este fichero contiene recursos precompilados, generalmente en formato XML binario.

- **assets/res**: Es el directorio donde podremos alojar los archivos auxiliares a la aplicación: imágenes, videos, audios… Estos recursos son los usados por la aplicación.

- **java/src**: Este el directorio que contiene porciones del código de la aplicación.

- **lib**: Este directorio puede contener código compilado, como por ejemplo, librerías precompiladas.

- **META-INF**: Este directorio contiene el fichero MANIFEST.MF, que almacena los metadatos sobre el contenido de los ficheros .jar. La firma de la aplicación APK también se almacena en esta carpeta.
- **CERT.RSA**: El certificado de la aplicación. CERT.SF: Listas de recursos necesarios y su correspondiente clave sha-1

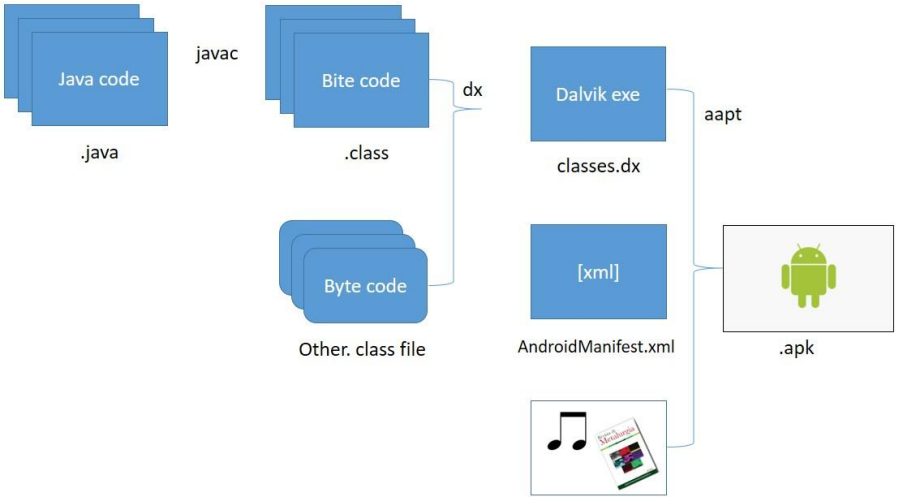


Imagen 6 **Estructura apk ampliada**

5.2. IOS

iOS es un sistema operativo móvil de la multinacional Apple Inc. Originalmente desarrollado para el iPhone (iPhone OS), después se ha usado en dispositivos como el iPod touch y el iPad. Apple no permite la instalación de iOS en hardware de terceros.

Este sistema operativo es considerado como uno de los más seguros debido a la gran cantidad de controles de seguridad que incorporan por defecto y que además no pueden ser modificados por el usuario.

Al igual que en Android, con el paso del tiempo y la inquietud de muchos investigadores se logró "vulnerar" la seguridad de los dispositivos de Apple con el fin de añadir funciones extra. A este proceso se le conoce como Jailbreak, ligado en muchos casos al tipo de dispositivo. Apple, en un intento de mantener cerrados y seguros sus dispositivos y su sistema operativo, persigue este tipo de modificaciones de manera bastante activa, forzando a los usuarios a actualizar sus dispositivos, con nuevas versiones y/o parches. La complejidad de análisis en aplicaciones IoS en mucho más alta que en Android.

iOS encripta todas las aplicaciones descargadas de la App Store utilizando su tecnología DRM llamada FairPlay. Para poder analizar el fichero final descifrado es necesario realizar modificación en un dispositivo Físico, e instalar diferentes herramientas para la transferencia y descifrado del fichero y su posterior análisis estático y dinámico. Concretamente iOS tiene que desencriptar primero la aplicación para que el sistema operativo pueda ejecutar el ejecutable real y herramientas como Clutch aprovecha eso para "desencriptar" la aplicación. En pocas palabras, lo que hace es "pedir" al sistema operativo que cargue la aplicación en la memoria para ejecutarla, y luego volcar la versión descifrada de la aplicación desde la memoria y escribirla en el disco.

Durante el curso no se cubrirán las acciones para descifrar aplicaciones y transferirlas a tu ordenador, ya que implica la necesidad de un dispositivos físicos sobre el que realizar Jailbreak.

Nos centraremos en realizar análisis en archivos. ipa ya descifrados.

5.2.1. LENGUAJE IOS

Para llevar a cabo el desarrollo de aplicaciones en iOS tan solo es necesario disponer de un ordenador Mac y el framework XCode instalado. En un principio, el lenguaje utilizado para crear aplicaciones en dispositivos de Apple fue Objective-C, aunque posteriormente desarrolló un nuevo lenguaje llamado Swift. En la actualidad ambos son compatibles para la creación de aplicaciones.

- **Objective-C** es un lenguaje de programación orientado a objetos creado como un superconjunto de C para que implementase un modelo de objetos parecido al de Smalltalk. Originalmente fue creado por Brad Cox y la corporación StepStone en 1980. En 1988 fue adoptado como lenguaje de programación de NEXTSTEP y en 1992 fue liberado bajo licencia GPL para el compilador GCC. Actualmente se usa como un lenguaje principal de programación para Mac OS X, iOS y GNUstep, además de Swift.

- **Swift** es un lenguaje de programación multiparadigma creado por Apple enfocado en el desarrollo de aplicaciones para iOS y macOS. Fue presentado en la WWDC 20147 y está diseñado para integrarse con los Frameworks Cocoa y Cocoa Touch; puede usar cualquier biblioteca programada en Objective-C y llamar a funciones de C. También es posible desarrollar código en Swift compatible con Objective-C bajo ciertas condiciones. Swift tiene la intención de ser un lenguaje seguro, de desarrollo rápido y conciso. Usa el compilador LLVM incluido en Xcode 6. Fue presentado como un lenguaje propietario, pero en el año 2015, con la versión 2.2, pasó a ser de código abierto8 con la Licencia Apache 2.0.

5.2.2. COMPONENETES IOS

iOS cuenta con cuatro capas de abstracción:

- **Núcleo del sistema operativo:** La capa del sistema operativo central contiene las características de bajo nivel sobre las que se basan la mayoría de las demás tecnologías.
    - Core Bluetooth Framework. o Marco de aceleración. o Marco de accesorios externos. o Marco de servicios de seguridad.
    - Marco de autenticación local.

- **Servicios Principales:** Se detallan algunos de los Frameworks importantes disponibles en las capas de servicios centrales: o Marco de la libreta de direcciones: Da acceso programático a una base de datos de contactos del usuario.
    - Cloud Kit framework: Da un medio para mover los datos entre su aplicación y iCloud.
    - Core data Framework: Tecnología para gestionar el modelo de datos de una app Model View Controller.
    - Core Foundation framework: Interfaces que ofrecen funciones fundamentales de gestión de datos y servicios para aplicaciones iOS.
    - Core Location framework: Proporciona información de ubicación y dirección a las apps.
    - Core Motion Framework: Accede a todos los datos basados en el movimiento disponibles en un dispositivo. Utilizando este marco de movimiento básico se puede acceder a la información basada en el acelerómetro.
    - Marco Homekit: Nuevo marco para hablar con los dispositivos conectados en el hogar del usuario y controlarlos.
    - Marco social: Interfaz sencilla para acceder a las cuentas de redes sociales del usuario.
    - Marco StoreKit: Da soporte a la compra de contenidos y servicios desde dentro de sus aplicaciones iOS, una característica conocida como InApp Purchase.
- **Medios**: La tecnología de gráficos, audio y vídeo se habilita mediante la capa de medios.
- **Cocoa Touch:** Cocoa Touch es un API para la creación de programas para el iPad, iPhone y iPod Touch de la compañía Apple Inc. Cocoa Touch proporciona una capa de abstracción al sistema operativo iOS. Algunos de sus componentes:
    - EventKit framework: da controladores de vista para mostrar las interfaces estándar del sistema para ver y alterar los eventos relacionados con el calendario
    - GameKit Framework: implementa el soporte para Game Center que permite a los usuarios compartir su información relacionada con el juego en línea
    - iAd Framework: le permite ofrecer anuncios basados en banners desde su aplicación.
    - MapKit Framework: proporciona un mapa desplazable que puede incluir en su interfaz de usuario de la aplicación.
    - PushKitFramework: proporciona soporte de registro para aplicaciones VoIP.
    - Twitter Framework: soporta una UI para generar tweets y soporte para crear URLs para acceder al servicio de Twitter.
    - UIKit Framework: proporciona una infraestructura vital para la aplicación de aplicaciones gráficas y basadas en eventos en iOS. Algunas de las funciones importantes de UI Kit framework:
        - Soporte multitarea.
        - Gestión e infraestructura básica de la app.

- Gestión de la interfaz de usuario.
- Soporte para eventos Touch y Motion.
- Soporte para cortar, copiar y pegar y muchas más.

5.2.3. ESTRUCTURA IOS

Al contrario que Android, IOS permite a los desarrolladores elegir la estructura de directorios y ficheros, con lo que no existe una estructura definida para el correcto funcionamiento de la aplicación.

El formato de archivos **.ipa** es el formato utilizado para las aplicaciones de Apple en los dispositivos iPhone, iPod Touch y iPad. Solo funciona en los dispositivos que usen iOS.

Este es un archivo de aplicación, básicamente un archivo zip. Incluye el binario de la aplicación, frameworks de terceros, archivos de configuración, archivos multimedia (como imágenes y vídeos), elementos de la interfaz de usuario (como storyboards y nibs), fuentes personalizadas y cualquier otro archivo que los desarrolladores incrusten en la aplicación.

El directorio Bundle o el contenedor IPA: consiste en todos los archivos que vienen junto con la aplicación cuando se instala desde la App Store de Apple o cualquier otra fuente. Por lo tanto, los archivos de este directorio seguirán siendo los mismos en una versión concreta de una aplicación. Se puede obtener mucha información importante sobre la aplicación a partir de los archivos de este directorio. A continuación, se explican todos los posibles componentes del directorio Bundle de una aplicación nativa de iOS:

Resource Name	Resource Type	Description
iTunesMetadata.plist	File	This file is used to provide information to iTunes about an iOS application using Ad Hoc distribution for either testing or Enterprise deployment.
iTunesArtWork	File	This file contains the image that is used to represent the application in iTunes
META-INF	Directory	It contains two files which hold the metadata about the IPA file.
.app Directory	Directory	This directory holds all the components of the IPA file.
Application Binary	File	This file contains the application's executable code. It's name is same as that of the name of .app Directory excluding the extension ".app".
Info.plist	File	This file is the manifest of the iOS application. It contains information about - Supported Devices, Bundle ID, Display Name, Application Transport Security etc.
Application Icon	File	These are the icon files of the application. There are multiple icon files (Icon.png, Icon@2x.png) for representation of the application on devices with different resolution like iPhone, iPad etc.
Launch Images	File	These files (Default.png, Default-portrait.png) are used as launch screen images before the application launches. They are removed as soon as the application is ready to display the UI.
Storyboard/nib files	File / Directory	These files contain the encrypted information about the storyboard of an application.

Imagen 7 **Tabla IPA contenidos**

Settings.bundle	File	This file contains the information that can be tweaked for the application by using the menu in the settings of the device.
Subdirectories for localized resources	Directory	These are the language-specific project directories (en.lproj, fr.lproj etc.) that help in switching the language of the application from the available languages as desired by the user of the application.
Non Localized resource files	File	These files include things like images, sound files, movies, and custom data files that the application uses. All these files must be placed within the .app directory.
_CodeSignature	Directory	This directory contains the file CodeResources which is used to store the signature of all files in the bundle that are signed.
embedded.mobileprovision	File	This plist file contains the provisioning profile for an application. In simple words, provisioning profile acts as a link between the device and the developer account. This file contains information such as application creation date, keychain access groups etc.
Frameworks	Directory	This directory holds the information about the frameworks that an application uses.
SC_Info	Directory	This directory contains keys which are used for decrypting the app executables. The files in this directory help in integrity checks of the application
Assets.car	File	This file contains images which the app might use to display once opened. The images are stored in an optimized format, and be only taken out using specialized tools such as the Asset Catalog Tinkerer.
PkgInfo	File	This file is an alternate way to specify the type and creator codes of the application or bundle. This file is not required, but is sometimes used to improve performance for code that accesses this information.

Imagen 8 **Tabla IPA contenidos**

El contenedor de datos

El directorio "Data" o el contenedor de almacenamiento de datos local consiste en los archivos que el desarrollador desea almacenar para la aplicación durante el tiempo que la aplicación está instalada en el dispositivo.

Los archivos pueden utilizarse para almacenar información en la memoria caché para un acceso rápido o para almacenar información fuera de línea como copia de seguridad también para reanudar la aplicación desde el punto previsto por el desarrollador. Por lo tanto, los archivos en este directorio y también la información en los archivos seguirá cambiando mientras la aplicación está en uso según lo codificado por los desarrolladores.

De estos archivos puede obtenerse información muy crítica sobre el usuario o la aplicación, dependiendo de la información que pretenda almacenar el desarrollador en este directorio. A continuación, se explican algunos de los posibles componentes del directorio "Data" de una aplicación iOS:

Parent Directory	Component Name	Description
Documents	NA	This directory is used to store the data that the user needs to access from the application and the files such as pdfs downloaded at the runtime. This may also be used to store crash logs that the user gets access to at the time of giving a feedback
Library	Application Support	This directory is used to store all app data files except those associated with the user's documents. Sometimes it may also be used to store a modifiable copy of resources contained initially in the app's bundle. The contents of this directory are backed up by iTunes and iCloud
	Caches	This directory is used to write any app-specific support files that the application can recreate easily. The data in this directory is mostly the cache for the analytics that can be sent when required and also the server's responses for delivering quick responses to the user's queries. This directory also stores the screenshot of the application in the Snapshots directory when it moves to the background in order to improve user experience.
	Preferences	This directory contains app-specific preference files. The main file in this directory is the file named <Bundle_ID>.plist which is used by the developers to store information using NSUserDefaults class.
tmp		This directory usually contains the backup data for an application or some other data that is intended to be deleted after a process gets over. The NSTemporaryDirectory function helps the developers write temporary data to this directory.
Storekit		This directory is important only for business perspective. It provides the access to the following: • **In-App Purchase -** Offers and promotes in-app purchases for content and services. • **Apple Music -** Checks a user's Apple Music
		capabilities and offers a subscription. • **Recommendations and reviews -** Provide recommendations for third-party content and enable users to rate and review your app.

Imagen 9 **Tabla IPA contenidos**

Contenedor iCloud

Este directorio contiene los datos que utilizan las aplicaciones de iOS habilitadas para iCloud. Los archivos de este directorio están destinados a ser almacenados y actualizados por las fuentes desde las que un usuario decide actualizar el archivo.

Normalmente consta de dos partes

- Documentos: Los archivos de este directorio están destinados a ser leídos y actualizados directamente por el usuario. Estos archivos son respaldados en iCloud regularmente para mantenerlos sincronizados.

- Datos: Estos archivos no están destinados a ser editados o añadidos directamente por el usuario. Los datos pueden guardarse en diferentes directorios según lo desee el desarrollador.

5.3. WINDOWS PHONE

Este sistema operativo es, quizá, el menos utilizado de los tres mencionados. Su cuota de mercado no ha tenido tanto éxito como Android o iOS, aunque en la actualidad cuenta con un gran número de dispositivos corriendo este sistema operativo. La extensión utilizada para la instalación de aplicaciones del sistema operativo de Microsoft es ".xap" y para el desarrollo de las mismas se puede utilizar el framework .NET. Al igual que con los sistemas anteriores, para Windows Phone también tenemos una serie de añadidos, fruto de la investigación de la comunidad. A este proceso se le conoce como Unlocking.

Al igual que con el sistema operativo de Apple, Microsoft ha cerrado las aplicaciones para sus dispositivos Windows Phone de manera "eficiente". Si bien no se pueden desempaquetar como las aplicaciones de Android, sí que es posible realizar un ataque de ingeniería inversa sobre la aplicación (al igual que sucede con las aplicaciones ".exe" en Windows). Actualmente no hay ningún framework o programa conocido que sea capaz de extraer información sobre el contenido de un fichero ".xap" (de manera similar a iOS).

El análisis estático de una aplicación desarrollada para Windows Phone se puede realizar en un dispositivo físico y no iría más allá de comprobar el comportamiento de la aplicación e intentar forzar algún tipo de error.

6. ENTORNO DE ANÁLISIS Y HERRAMIENTAS

Hablemos en primer lugar del entorno de análisis. Es posible montar un entorno de análisis completo tanto en un sistema host Windows como en un Linux o Mac. Es también importante saber que la instalación del entorno en un Host físico de Linux será más "directa" dado que la mayoría de herramientas están pensadas para ser utilizadas en este entorno. Es posible también hacerlo en Windows, pero deberemos instalar una serie de herramientas para poder hacerlo. ¿Por qué no usamos una MV Linux para instalarlo todo allí? Porque algunas de las herramientas que han de instalarse para emular los dispositivos corren sobre una MV, así que no podremos tener una MV dentro de otra. A continuación, se muestra un posible entorno basado en un host Windows:



Imagen 10 **Esquema arquitectura entorno de análisis Windows host**

Para llevar a cabo un análisis completo necesitamos, en primer lugar, un elemento que sea capaz de reproducir el soporte en el que las aplicaciones pueden ejecutarse. Las herramientas que realizan estas acciones se llaman emuladores. No todos los sistemas operativos móviles pueden ser emulados. Para este caso necesitaremos un dispositivo físico: tablet, smartphone, etc. Durante el curso utilizaremos únicamente dispositivos emulados Android con software específicamente diseñado para tal efecto, dado que los emuladores de IoS aun están muy verdes para nuestro propósito.

Si se trata de un sistema operativo windows, tendremos que poner especial atención a la instalación de Python y de todas las dependencias, ya que la mayoría de software que utilizaremos esta escrito en este lenguaje y suele estar pensado para entornos Linux. Será más sencillo si tenemos instalado también software como Git o Pip.

En el caso de un Linux, la instalación suele ser más sencilla porque la documentación y las propias herramientas suelen estar pensada para este tipo de sistema operativo. Utilizaremos una MV Kali, con todas las herramientas requeridas instaladas para los casos en que no podamos utilizar Windows.

Existen muchas herramientas que ofrecen el mismo resultado, así que veremos algunas de ellas y proporcionaremos un listado completo con el resto. Utilizaremos otras otras herramientas más generales y también Frameworks completos como MobSF, que nos ayudarán a analizar una herramienta y obtener sus vulnerabilidades, códigos intermedios, etc las cuales nos ayudarán a no perder tiempo aprendiendo varias herramientas y usar una que cubra el espectro más amplio de tareas.

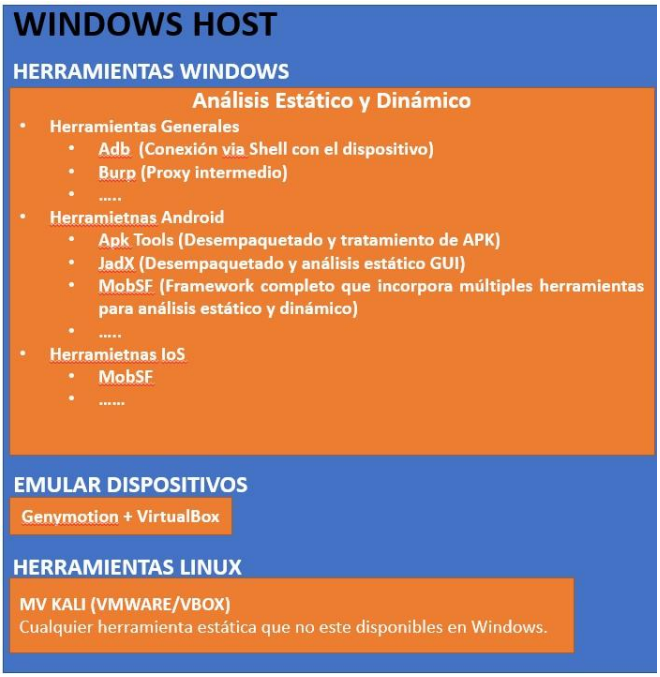


Imagen 11 **Esquema entorno de análisis Windows host**

Dado que cada alumno dispondrá de una infraestructura concreta, no se pueden cubrir las instalaciones de cada posible entorno durante el curso. La instalación de todas las herramientas necesarias está bien documentada en su página oficial. Son gratuitas y sólo en el caso de Genymotion será necesario que cada alumno registre una cuenta para poder usarla en su modalidad de uso particular. Utilizaremos el foro como punto de encuentro común para resolver los posibles problemas que puedan aparecer durante el despliegue del entorno de análisis.

Problemas generales:

- Hay que hacer siempre un update y upgrade en Linux antes de comenzar. Sino podemos tener problemas de dependencias.
- NO instalar Docker en la MV host Windows si se quiere disponer de Virtualización. Se producen problemas por la activación de HyperV que Docker realiza al instalarse.
- Puede ser necesarias dos versiones simultaneas de Python en Windows. Debemos añadir al path de sistema ambas.

6.1. HERRAMIENTAS ANDROID.

Para llevar a cabo el análisis de aplicaciones Android necesitamos configurar y levantar un entorno controlado (máquina virtual) con todas las herramientas necesarias.
Algunas de ellas tienen la misma finalidad, pero vamos a utilizar varias para recopilar el mayor número de datos posibles. Las herramientas son:

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

- Máquina virtual con la última versión de Kali Linux (recomendable como mínimo 4 gigas de memoria RAM y 2 procesadores), en su versión de 64 bits.
- ADB: Se trata de un sistema para comunicarte y controlar un dispositivo Android mediante línea de comandos.
- BurpSuite o ZAP – Proxy. (Opcional)
- Wireshark – Sniffer de red. Viene instalado por defecto en Kali Linux 2
- Genymotion: Emulador de Android
- VirtualBox: Software para crear máquinas virtuales
- MobSF – Framework para análisis de aplicaciones Android

6.2. HERRAMIENTAS NECESARIAS IOS.

Dado que únicamente realizaremos análisis estático al no disponer de dispositivos móviles, utilizaremos básicamente MobSF para realizar el análisis. También utilizarémos GHidra para el análisis estático avanzado.

7. PUNTOS DE ENTRADA ANDROID

Uno de los puntos más importantes de la ingeniería inversa es saber por dónde empezar el análisis. Veremos como los puntos de entrada de la ejecución del código son una parte importante a la hora de comenzar el análisis.

7.1. ARCHIVO MANIFIEST

Vimos anteriormente el contenido de este fichero y los datos de interés que podremos obtener, por lo tanto, será un punto de partida.

7.2 PERMISOS INNECESARIOS

Una aplicación puede solicitar permisos que no son esenciales para la funcionalidad de la aplicación y pueden ser utilizados maliciosamente por cualquier otra aplicación o servicio en el dispositivo. Los encontraremos en el archivoManifiest.

7.3 DATOS SENSIBLES EN LAS CADENAS DE TEXTO

La aplicación puede contener cadenas sensibles que podrían indicar a un atacante cosas como la lógica de cifrado/descifrado, contraseñas, etc. Las cadenas se pueden comprobar utilizando diferentes herramientas.

7.4 SEGURIDAD DE TRANSPORTE

Una aplicación puede estar mal configurada para permitir la carga de URLs arbitrarias en la aplicación, lo que puede causar problemas de seguridad provocando un comportamiento malicioso en la aplicación.

7.5 ACTIVIDAD DEL LANZADOR

La actividad del lanzador es la actividad que se inicia cuando un usuario hace clic en el icono de una aplicación. En el **fichero manifiest** de la aplicación podemos encontrar los datos que nos interesan. La actividad del lanzador tendrá los siguientes Intents MAIN y LAUNCHER listados en el fichero.

El siguiente código fuente muestra cómo declarar que una actividad de Android es la actividad de lanzamiento de su aplicación:

```
<activity android:name=".MyMainActivity" >
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
<activity
    android:name=".SomeOtherActivity"
```

```
    android:label="@string/app_name" >

</activity>
```

Este código debe encontrarse en el fichero AndroidManifest.xml, y declara que una clase Java llamada MyMainActivity es la actividad de lanzamiento para la aplicación Android.

En ocasiones las aplicaciones no tendrán una actividad de lanzamiento, esto sucede generalmente en las aplicaciones sin interfaz de usuario. Generalmente son las aplicaciones preinstaladas que realizan servicios en segundo plano, como el buzón de voz. El siguiente código muestra un ejemplo:

```
<activity android:name=".LauncherActivity">
            <intent-filter>
        <action android:name="android.intent.action.MAIN" />
      <category android:name="android.intent.category.LAUNCHER" />
   </intent-filter>
</activity>
```

7.6 SERVICIOS

Los servicios se ejecutan en segundo plano sin una interfaz de usuario. Hay un sinfín de formas en las que pueden iniciarse y, por tanto, son un punto de entrada para las aplicaciones. La forma predeterminada en que un servicio puede iniciarse como punto de entrada a una aplicación es a través de Intents.

Cuando se llama a la API startService para iniciar un servicio, se ejecuta el método onStart del servicio.
Mas información sobre servicios en: **Enlace**

7.7 BROADCAST RECEIVERS

Pueden considerarse un sistema de mensajería y los receptores de emisiones son los oyentes. Si una aplicación ha registrado un receptor para una emisión específica, el código de ese receptor se ejecuta cuando el sistema envía la emisión. Hay dos maneras en que una aplicación puede registrar un receptor: en el Manifiesto de la aplicación o registrado dinámicamente en el código de la aplicación utilizando la llamada a la API registerReceiver().

En ambos casos, para registrar el receptor, se establecen los filtros de intención para el receptor. Estos filtros de intención son las emisiones que deben activar el receptor.

Cuando se envían las emisiones específicas para las que está registrado el receptor, se ejecuta onReceive en la clase BroadcastReceiver.

7.8 COMPONENTES EXPORTADOS (SERVICIOS Y ACTIVIDADES)

Los servicios y las actividades también se pueden "exportar", lo que permite que otros procesos del dispositivo inicien el servicio o lancen la actividad. Los componentes se exportan estableciendo un elemento en el manifiesto como el siguiente. Por defecto, android:exported="false" a menos que este elemento se establezca como verdadero en el manifiesto o se definan filtros de intención para la Actividad o el Servicio.

```
<service android:name=".ExampleExportedService" android:exported="true"/>

<activity android:name=".ExampleExportedActivity" android:exported="true"/>
```

7.9 SUBCLASE DE APLICACIÓN

Las aplicaciones de Android pueden definir una subclase de Aplicación. Las aplicaciones pueden, pero no tienen que definir una subclase personalizada de Application. Si una aplicación Android define una subclase de Application, esta clase se instanciará antes que cualquier otra clase de la aplicación.

Si el método attachBaseContext se define en la subclase Application, se llama primero, antes del método onCreate.

8. PUNTOS DE ENTRADA IOS

Tal y como hemos comentado centraremos nuestro análisis en los ficheros .ipa. A continuación, veremos los puntos donde analizar e identificar posibles vulnerabilidades de seguridad en los archivos presentes en el archivo IPA de una aplicación iOS.

8.1. ARCHIVO INFO.PLIST

Este archivo es el manifiesto de una aplicación iOS y se utiliza para almacenar los datos de configuración de la aplicación. Es un archivo de lista de propiedades de Apple. Determina qué icono mostrar para un paquete, qué tipos de documentos admite una aplicación y muchos otros comportamientos que tienen un impacto fuera del propio paquete.

Leyendo el archivo plist, podemos reunir alguna información básica sobre la aplicación como:

- Información de la
- Nombre de la aplicación
- Nombre del directorio .app de la
- Identificador de paquete de la aplicación
- Nombre binario ejecutable
- Versión de iOS y dispositivos compatibles
- Nombres de archivos de imagen de iconos y de inicio
- Permisos de la aplicación
- Seguridad de transporte de la aplicación

8.2. PERMISOS INNECESARIOS

Una aplicación puede solicitar permisos que no son esenciales para la funcionalidad de la aplicación y pueden ser utilizados maliciosamente por cualquier otra aplicación o servicio en el dispositivo. La habilitación de permisos puede hacer que el usuario haga algunas cosas no deseadas, como compartir la ubicación con terceros remotos, o puede utilizarse para actividades de ingeniería social mediante el uso de componentes del dispositivo como el micrófono, la cámara, etc.

8.3. DATOS SENSIBLES EN LAS CADENAS DE TEXTO

La aplicación puede contener cadenas sensibles que podrían indicar a un atacante cosas como la lógica de cifrado/descifrado, contraseñas, etc. Las cadenas se pueden comprobar ejecutando el comando UNIX 'strings' en el binario de la aplicación.

8.4. SEGURIDAD DE TRANSPORTE

Una aplicación puede estar mal configurada para permitir la carga de URLs arbitrarias en la aplicación, lo que puede causar problemas de seguridad provocando un comportamiento malicioso en la aplicación.

8.5. ANÁLISIS DEL BINARIO DE LA APLICACIÓN

El binario de una aplicación iOS es un archivo de tipo binario Mach-O. Es el archivo ejecutable real o el código máquina que se ejecuta en un iPhone. Utiliza los archivos del Bundle Container y produce lo que es visible para el usuario de la aplicación. La ingeniería inversa completa de una aplicación iOS para producir el código fuente no es posible. Pero se pueden volcar partes específicas de archivos de objetos o bibliotecas utilizando las herramientas existentes. Desde el punto de vista de la seguridad, no necesitaríamos ver toda la información que obtenemos de estas herramientas. Sólo buscaríamos ciertas palabras clave en la salida.

8.6. GENERADOR DE NÚMEROS ALEATORIOS INSEGURO

Las APIs como _rand, _srand y _random se consideran inseguras para generar números aleatorios ya que los resultados pueden ser predichos por un atacante. La complejidad del ataque es muy alta y además muchas veces los desarrolladores combinan estas APIs con otras criptográficamente fuertes. Así que la vulnerabilidad entra en la categoría de información.

8.7. USO DE ALGORITMOS HASH DÉBILES

Las APIs como MD5 y SHA1 se consideran algoritmos de hashing débiles debido a los problemas de colisión. Pueden ser descifrados usando sitios web que utilizan grandes bases de datos o usando una herramienta como hashcat. El impacto de la vulnerabilidad depende de dónde se esté utilizando la API. Si estas APIs están siendo usadas para hash de información sensible, un atacante puede intentar crackear esos hashes y obtener información crucial sobre los recursos del usuario o de la aplicación y entonces el impacto causado a la aplicación o al usuario aumentará. Muchas veces los desarrolladores combinan estas APIs con APIs criptográficamente fuertes haciendo un hallazgo de nivel bajo o informativo.

8.8. USO DE BANNED/DEPRECATED APIS

En Objective C, las APIs como strlen, memcpy, strcpy, etc. han sido desaprobadas ya que podrían llevar a la corrupción de memoria y no deberían ser utilizadas más. La vulnerabilidad puede ser explotada causando un desbordamiento de búfer que es muy difícil de explotar en el ecosistema iOS. Sin embargo, se considera una "mejor práctica" no utilizarlos.

Si un atacante es capaz de encontrar puntos de entrada a estas funciones, puede intentar enviar entradas maliciosas en tiempo de ejecución e intentar explotar la vulnerabilidad provocando un Buffer Overflow. Pero teniendo en cuenta la complejidad del ataque y las diversas medidas de seguridad de la plataforma iOS, la vulnerabilidad entra en la categoría de información.

8.9. BINARIO DE APLICACIÓN SIN CIFRAR

Siempre que una aplicación se sube a la App Store, viene con el cifrado Fairplay DRM de Apple. Pero es importante comprobar esto también.

8.10. FLAG DE COMPILACIÓN

- **Binario de aplicación compilado sin la bandera PIE**

ASLR (Address Space Layout Randomization) protege el binario de la aplicación iOS contra las vulnerabilidades de corrupción de memoria mediante la aleatorización de la ubicación de los objetos de la aplicación en la memoria cada vez que la aplicación se reinicia. Se implementa compilando el binario de la aplicación iOS con la bandera PIE (Position Independent Executable).

Si ASLR no está habilitado para una aplicación iOS, el desplazamiento de la ubicación de los módulos y de ciertas estructuras en memoria no tendría lugar de forma aleatoria y, por lo tanto, abriría una puerta para un ataque de desbordamiento del búfer.

- **Binario de aplicación compilado sin la bandera fobjc-arc flag**

ARC (Automatic Reference Counting) ayuda a la gestión automática de la memoria en las aplicaciones iOS mediante el manejo del recuento de referencias de los objetos de forma automática en tiempo de compilación. Se implementa compilando el binario de la aplicación iOS con la bandera fobjc-arc.
Desactivar ARC para una aplicación iOS estabilizará el recuento de referencias para los objetos y dará al atacante la oportunidad de corromper la memoria del dispositivo de la víctima y también explotar la vulnerabilidad de desbordamiento del búfer.

- **Binario de aplicación compilado sin la bandera fstack-protector-all**

La protección contra el aplastamiento de la pila está implícita en una aplicación iOS colocando un valor conocido o "canario" en la pila directamente antes de las variables locales para proteger el puntero base guardado, el puntero de instrucción guardado y los argumentos de la función. El valor del canario se comprueba en el evento de retorno de la función y se informa si hay algún cambio.
Si la protección contra el aplastamiento de la pila no está habilitada, el atacante trataría de insertar cargas útiles maliciosas en la aplicación y, por lo tanto, llevaría a la caída de la aplicación en tiempo de ejecución haciendo que la experiencia del usuario para la aplicación sea mala. La vulnerabilidad es difícil de atacar, pero no imposible.

9. COMPRENSIÓN DE LA APLICACIÓN

Para comprender la aplicación habitualmente la instalaremos en el emulador y la ejecutaremos dinámicamente. Habremos mirado previamente el archivo manifiest.xml o comprobaremos

posibles cifrados entre otras posibles acciones de análisis estático que nos ayuden a mapear la aplicación.

Recordemos que los objetivos son los siguientes:

- **Entender funcionamiento de la app a nivel usuario**
- **Entender su lógica, usuarios y privilegios**
- **Cuál es el flujo de la comunicación entre objetos**
- **Cuándo se produce una comunicación con el servidor**
- **Atención especial a eventos de la aplicación Infraestructura de la aplicación**

9.1. COMPRENSIÓN DE LA APLICACIÓN EN LA PRÁCTICA

El escenario del análisis y la infraestructura son:

Windows 10 maquina host con el software instalado:

- Gennymotion
- VirtualBox
- Adb.exe

En este punto lo primero que hemos de hacer es instalar la herramienta en nuestro en nuestro entorno Android emulado.

Para ello en primer lugar abriremos Genymotion y lanzaremos un dispositivo Android.



Imagen 12 **Genymotion Android vm**

Seguidamente con el programa adb.exe listaremos en primer lugar los dispositivos disponibles mediante el comando "**adb devices**" y seguidamente instalaremos la aplicación usando el comando "**adb install diva-beta.apk**".

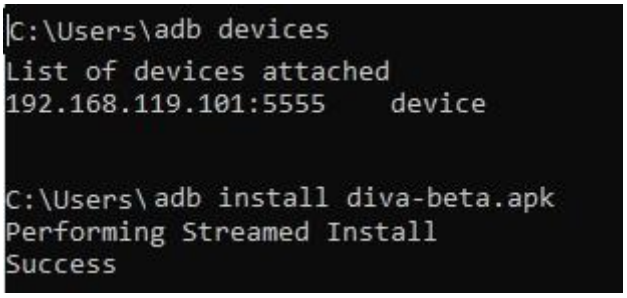


Imagen 13 **programa adb**

Una vez instalado podemos encontrar en nuestro dispositivo la aplicación y podemos abrirla.

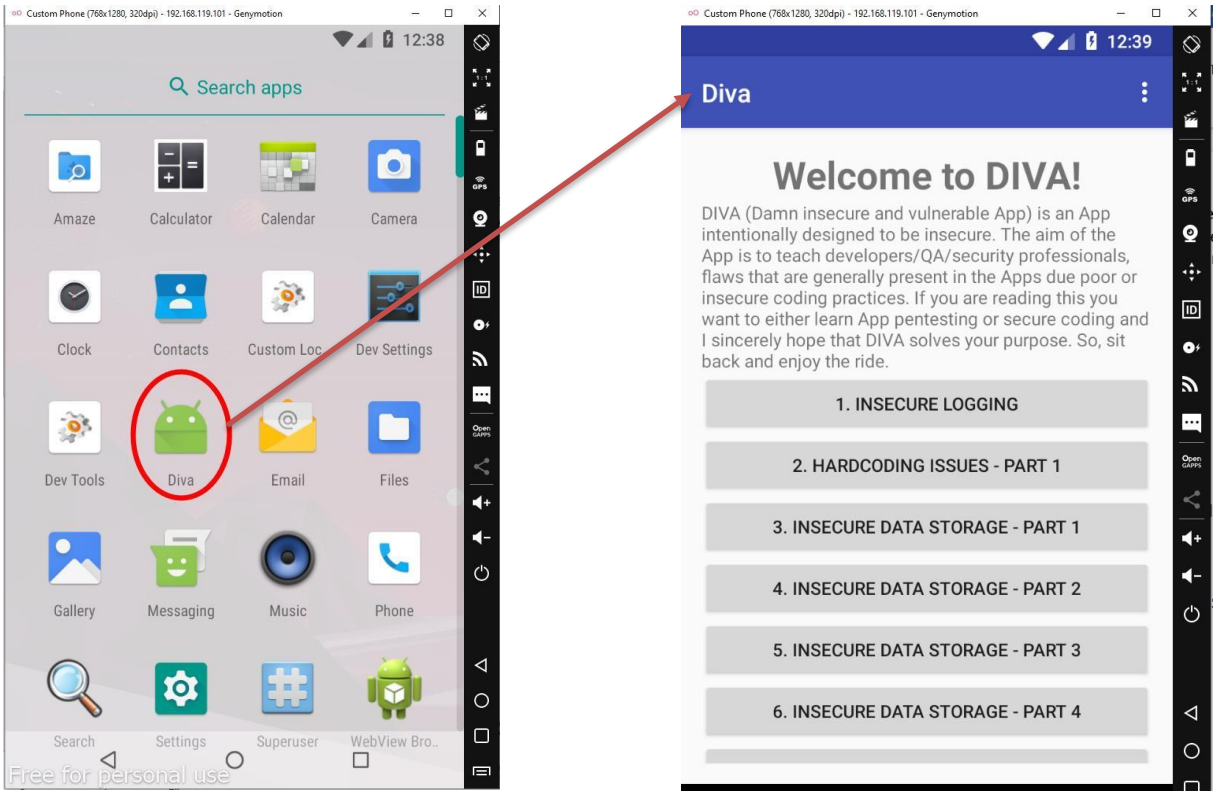


Imagen 14 **DIVA aplication**

Ahora ya solo se trata de jugar con la aplicación para entender cuales es su funcionamiento, cuáles son sus parámetros de entrada, es decir, donde puedo introducir un dato, el cual será tratado internamente en la aplicación como una variable y puede ser utilizado de diferentes formaas para tomar ventaja de un código que no haya limpiado correctamente la variable.

Debemos identificar puntos de comunicación con un servidor, cifrado y cualquier otro detalle que sea de interés para proceder después con más detalle y precisión en las siguientes fases del análisis.

10. ANÁLISIS ESTÁTICO

10.1. ¿QUÉ ES?

Este tipo de análisis se llevan a cabo sobre el fichero de la aplicación (los .apk en caso de Android y los .ipa en caso de iOS) y consiste en extraer la máxima cantidad de información posible de las aplicaciones para después analizarla dinámicamente.

Durante el módulo realizaremos análisis estáticos de las aplicaciones de iOS y Android.

Recordemos que los principales objetivos de esta fase son:

- **Información genérica del binario**
- **Información de sub-binarios y arquitecturas**
- **Volcado de clases**
- **Análisis de símbolos**
- **Análisis de dependencias**
- **Análisis de strings**
- **Comprobación de cifrado de la aplicación**
- **Comprobación de activación de ASLR (PIE)**
- **Desensamblado**
- **Depuración**

10.2. DESEMPAQUETADO DE LA APLICACIÓN

El desempaquetado de una aplicación consiste en abrirla, simplemente, intentando modificar lo menos posible la misma, para que no sufra alteraciones. El objetivo es obtener una versión lo más fidedigna posible del código de la aplicación, para proceder a su análisis en busca de fallos, vulnerabilidades, cadenas de texto, funciones peligrosas…

A la hora de llevar a cabo el análisis de una aplicación móvil tenemos que seguir una serie de pautas para comprobar e intentar entender su comportamiento: analizando el propio fichero de la aplicación y su comportamiento en un entorno controlado. Ambas reciben el nombre de análisis estático y análisis dinámico respectivamente.

Para realizar análisis estáticos lo único que necesitamos es el propio fichero de la aplicación. .Apk en caso de Android y los .ipa en caso de IoS. En ambos casos con descomprimirlos como si fuera un fixhero zip será suficiente. El resto de tareas posteriores los realizarán herramientas por nosotros de forma automática y básicamente tratarán de mostrar el decompilado de la aplicación.

10.3. ANÁLISIS ESTÁTICO EN ANDROID

En el caso de las aplicaciones Android, el primer paso que tenemos que realizar es el desempaquetado de la aplicación. De esta forma tendremos acceso a parte del código y de la lógica de la aplicación. Vamos a analizar su contenido y veremos cómo podemos extraer información valiosa: permisos que necesita la aplicación, estructura de la misma, recursos disponibles, librerías...

10.3.1. ¿DONDE REALIZAMOS EL ANÁLISIS?

Una de las principales claves de la ingeniería inversa, independientemente de la plataforma, es saber por dónde empezar su análisis. Generalmente en análisis estático es una buena aproximación para tener una visión general. Pero ¿por dónde comenzar el análisis estático? Nos haremos esta pregunta, porque como sucede generalmente, los códigos de las aplicaciones son muy extensos y Android no es una excepción, así que nos centraremos en priorizar, dado que revisar línea a línea no suele ser un objetivo realista.

10.3.1.1. LLAMADAS API

Generalmente en las llamadas a la API veremos la mayoría de los comportamientos interesantes que podemos querer identificar en Android.

Por ejemplo, digamos que estás evaluando una aplicación para ver si está haciendo Fraude SMS Premium. Fraude SMS Premium significa que una aplicación está enviando un mensaje SMS Premium sin el consentimiento del usuario. Por lo tanto, para realizar el fraude, la aplicación debe enviar un mensaje SMS. Hay un número finito de llamadas a la API que permiten a una aplicación enviar un mensaje SMS. Por ejemplo, sendTextMessage, sendMultipartMessage,smsto:. Por lo tanto, uno de los puntos clave para comenzar el análisis es encontrar las llamadas a la API necesarias para el comportamiento que te interesa, y luego buscarlas en tu aplicación. A continuación, puede comenzar su inversión en torno a esas llamadas de la API.

10.3.1.2. PUNTOS DE ENTRADA DE LA APLICACIÓN

En muchos casos, sólo interesa el código que se puede ejecutar, no el código muerto, de la aplicación. Por lo tanto, empezar en un punto de entrada de la aplicación (detallado en esta sección en el punto 8) es una buena opción si no estás seguro de por dónde empezar.

10.3.1.3. MÉTODOS DE DESCIFRADO

Java depende en gran medida de las cadenas para realizar muchas de sus operaciones. Por ejemplo, para enviar intents o llamar a métodos mediante reflexión. Si su aplicación no tiene cadenas legibles para los humanos, entonces probablemente significa que está ofuscada o encriptada. Un buen punto de partida seguir el flujo de esas cadenas cifradas para buscar los

métodos que las utilizan. Esos métodos son probablemente los métodos de desofuscación o descifrado.

10.4. ANÁLISIS ESTÁTICO EN LA PRÁCTICA

La aplicación elegida para ser analizada está diseñada con vulnerabilidades, de forma deliberada, con el fin de proporcionar una aplicación y un entorno de "entrenamiento".

En este caso la aplicación sobre la que vamos a realizar el análisis es:
Descarga Aplicación

Utilizaremos diferentes herramientas para el análisis estático, que nos ayudarán en esta fase.

10.4.1. JADX-GUI

La forma más sencilla inicialmente será utilizar una herramienta con entorno gráfico que nos permita examinar el código y los recursos. En este caso utilizaremos Jadx, aunque hay alternativas muy interesantes como JD-Gui.

Una vez descargado el fichero del ejercicio, descomprimido y abierto el ficheo apk en la aplicación, vemos información clasificada en los diferentes componentes que observamos en la arquitectura de este tipo soluciones. Analizaremos y extraeremos la información que nos interesa de cada parte.



Imagen 15 **Estructura APK obtenida en jadx-Gui**

Como vemos encontraremos información relativa a los ficheros de interés como el AndroidManifiest.xml o clases.dex. Tendremos acceso tanto al código fuente como a los certificados. Ahora es el momento en el que buscaremos strings de interés que nos ayuden a buscar vulnerabilidades u otros indicios que nos ayuden a entender mejor la aplicación.

Buscaremos la cadena de texto “insert” ya que nos permitirá ver posible interacción con BD.

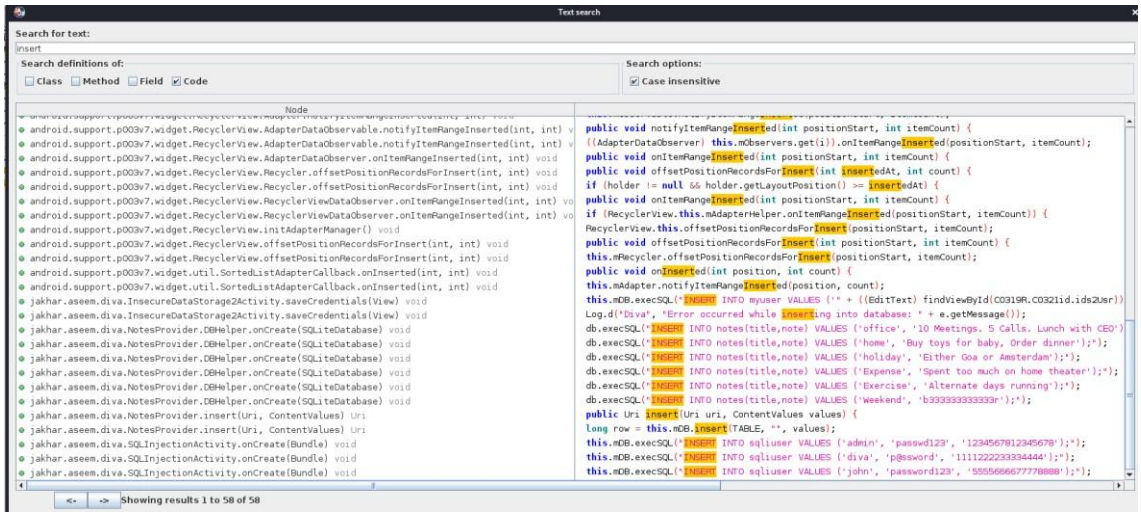


Imagen 16 **Búsqueda de Strings en con jadx-Gui**

En el detalle de esa captura vemos los nombres de usuario y contraseña harcodeados.

```
this.mDB.execSQL("INSERT INTO myuser VALUES ('" + ((EditText) findViewById(C0319R.C0321id.ids2Usr)
Log.d("Diva", "Error occurred while inserting into database: " + e.getMessage());
db.execSQL("INSERT INTO notes(title,note) VALUES ('office', '10 Meetings. 5 Calls. Lunch with CEO'
db.execSQL("INSERT INTO notes(title,note) VALUES ('home', 'Buy toys for baby, Order dinner');");
db.execSQL("INSERT INTO notes(title,note) VALUES ('holiday', 'Either Goa or Amsterdam');");
db.execSQL("INSERT INTO notes(title,note) VALUES ('Expense', 'Spent too much on home theater');");
db.execSQL("INSERT INTO notes(title,note) VALUES ('Exercise', 'Alternate days running');");
db.execSQL("INSERT INTO notes(title,note) VALUES ('Weekend', 'b333333333333r');");
public Uri insert(Uri uri, ContentValues values) {
long row = this.mDB.insert(TABLE, "", values);
this.mDB.execSQL("INSERT INTO sqliuser VALUES ('admin', 'passwd123', '1234567812345678');");
this.mDB.execSQL("INSERT INTO sqliuser VALUES ('diva', 'p@ssword', '1111222233334444');");
this.mDB.execSQL("INSERT INTO sqliuser VALUES ('john', 'password123', '5555666677778888');");
```

Imagen 17 **Strings APK obtenida en jadx-Gui**

Podemos hacer doble click en la función que os interesa y de este modo navegaremos a la sección de código en la que se utiliza.



Imagen 18 **Strings APK obtenida en jadx-Gui**

Podremos analizar la función concreta de código para entender que sucede.

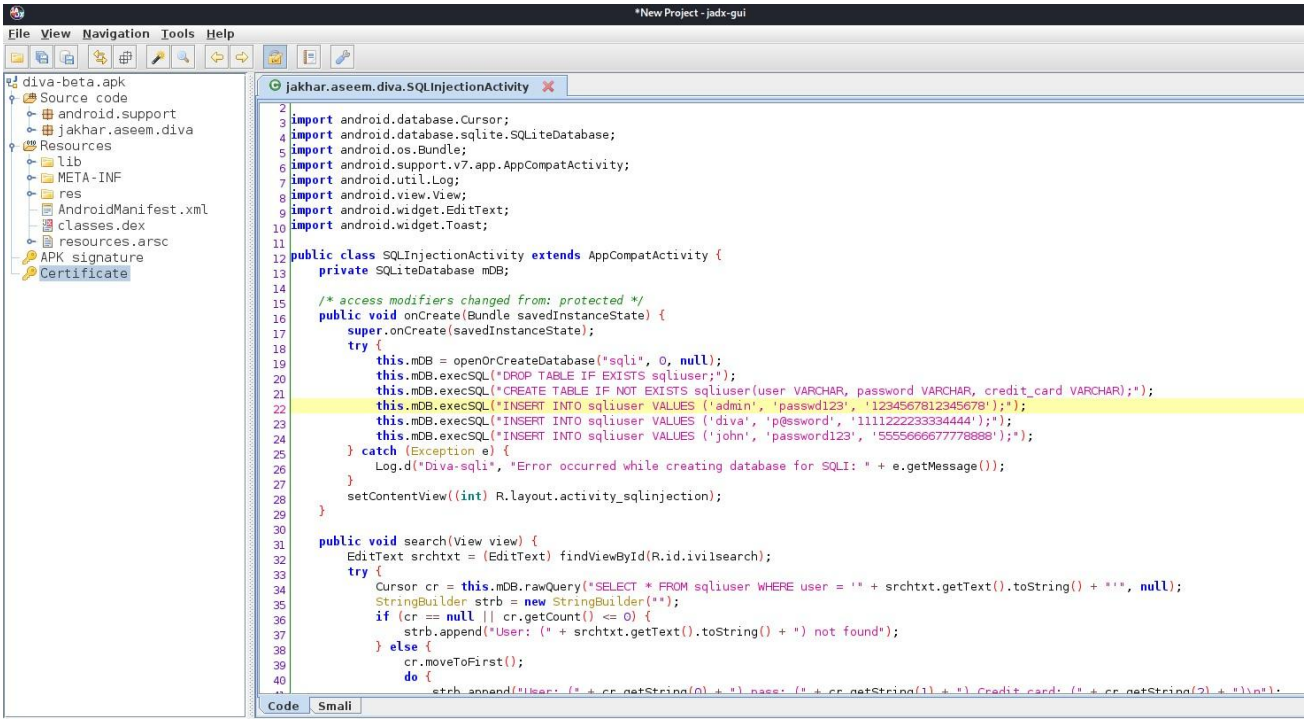


Imagen 19 **Código java de APK en jadx-Gui**

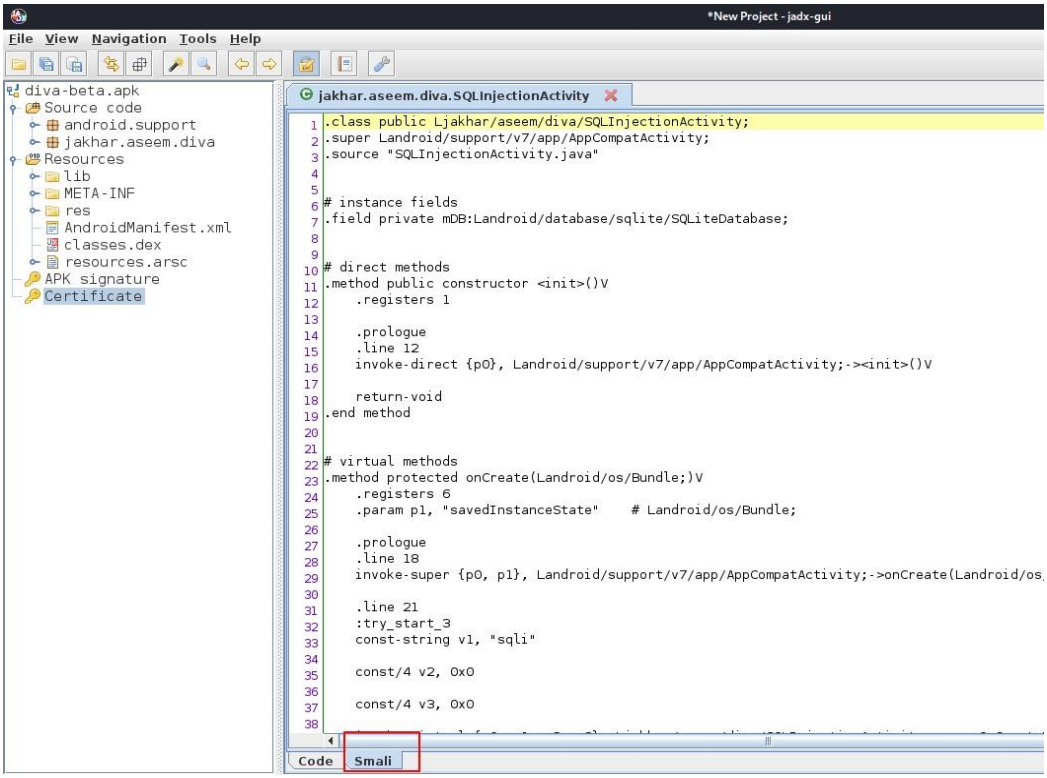


Imagen 20 **Código Smali de APK en jadx-Gui**

Podremos buscar con Ctrl+F dentro del código Smali, en nuestro caso buscaremos "Insert" para ubicar la misma sentencia de código que teníamos en java. En ocasiones no se decompilará bien el java y la comparación línea a línea puede ayudar a entender mejor Smali.

10.4.2. APKTOOL

Tambien vamos a utilizar la aplicación **apktool** para realizar el desempaquetado de la aplicación de forma más manual y controlada, obteniendo como resultado el desempacado de la aplicación con su estructura y ficheros correctamente creados.

Vamos a decirle a la aplicación que nos muestre información acerca del desempaquetado de la aplicación, con el parámetro **v** para que sea mas verbosity y con el parámetro **d** (función debug), seguido del nombre del fichero:

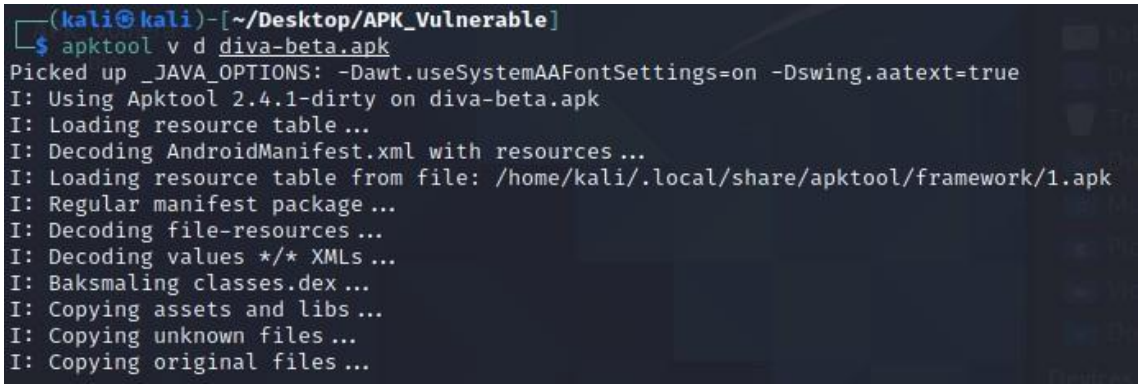


Imagen 21 **Proceso de desempaquetado apktool**

A continuación, nos habrá creado un directorio con todos los recursos disponibles de la aplicación:

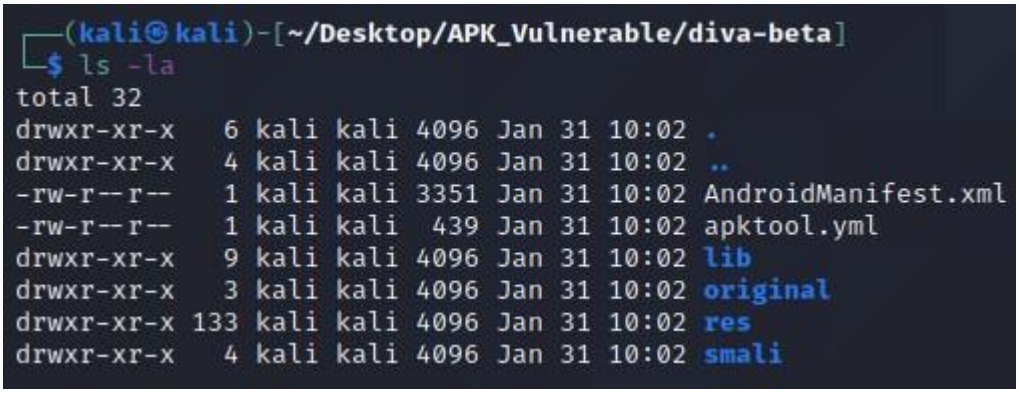


Imagen 22 **Resultado proceso desempaquetado apktool**

Podemos ver cómo nos ha desglosado la aplicación en diferentes ficheros y directorios. En este primer paso ya tendremos acceso a fichero **AndroidManifest.xml y la propia aplicación apktool**

se habrá encargado de "traducir" los ficheros. dex a ficheros legibles y, por lo tanto, podremos buscar cadenas de texto "interesantes":
nombres de usuario, contraseñas, sentencias sql...

Procedemos a abrir el fichero AndroidManifest.xml y podremos leer los primeros datos. Podemos ver el contenido completo del fichero, en texto legible y siguiendo las pautas recomendadas por el equipo de desarrollo de Android:

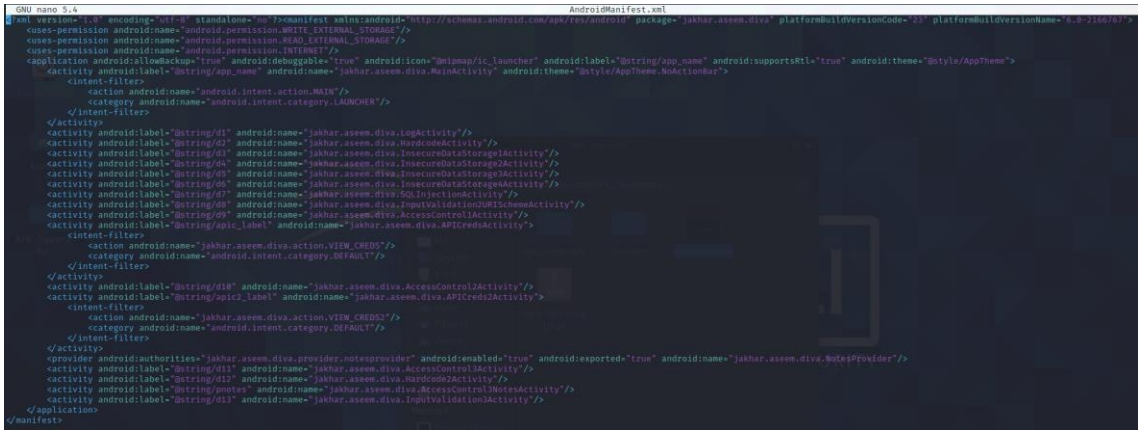


Imagen 23 **Contenido fichero AndroidManifest.xml**

En la siguiente captura, vemos los permisos de la aplicación (escritura, lectura e internet). También podemos ver como la aplicación tiene la opción "debuggable" con el valor "true". No será siempre así y en ocasiones habrá que modificar el valor y volver a firmar el fichero con herramientas específicas.

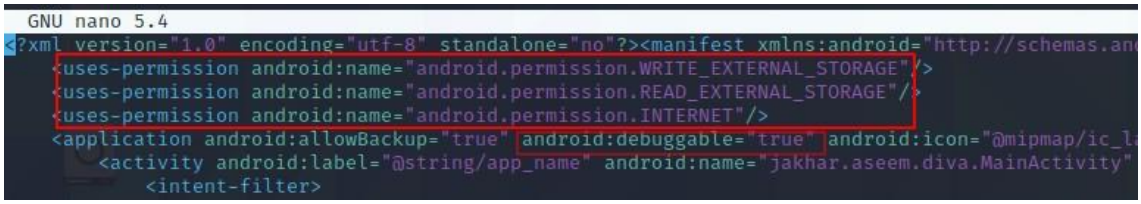


Imagen 24 **Permisos en aplicación y flag debug**

Esta aplicación nos ha facilitado el trabajo y nos ha convertido los ficheros que en un principio serían ilegibles en ficheros completamente legibles, donde utilizando una serie de comando en linux, podemos extraer información buscando sobre dichos ficheros.

En este caso hemos realizado una búsqueda de términos relacionados con el lenguaje SQL (con el fin de detectar consultas que la aplicación pueda realizar contra el servidor). Como se puede observar, hemos encontrado clausulas "INSERT INTO", con una serie de datos grabados en el código fuente del programa y que nos ayuda a pensar que pueden ser cadenas de usuarios y contraseñas:

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

Con la siguientse consulta buscaremos que ficheros smali contienen información de nuestro interés.



Imagen 25 **Búsqueda de cadena INSERT INTO**

Como resultado obtenemos

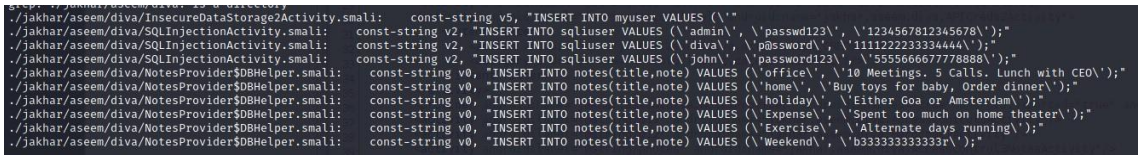


Imagen 26 **Cadenas con información sensible**

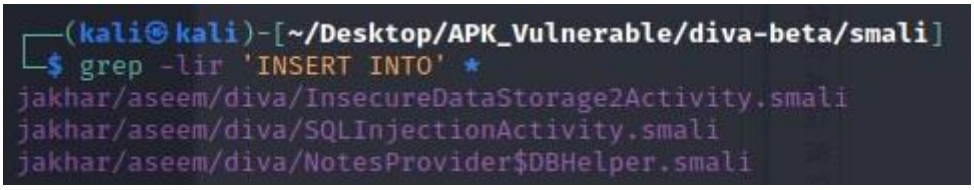


Imagen 27 **Búsqueda de cadena INSERT INTO**

Como vemos nos indica que ficheros Smali podrían ser interesantes al contener la consulta de inserción en BD “INSERT INTO”

10.4.3. MOBSF

En este caso vamos a utilizar el framework MobSF, en su última versión. Esta herramienta nos facilita el trabajo de análisis, puesto que levanta un servicio web al cual podemos acceder desde el navegador y podemos realizar todo el análisis en un entorno visual. En realidad, es una herramienta mucho más completa que nos ayudará en todas las partes del análisis, pero nos centraremos en la parte estática.

En primer lugar, necesitamos iniciar el framework. Para ello necesitamos situarnos, mediante la consola de linux, en el mismo directorio donde lo tenemos instalado y posteriormente iniciar la aplicación con el siguiente comando:
“**sudo ./run.sh**”
También tenemos la opción de facilitar la dirección ip (ha de ser la de la máquina virtual donde estamos) y el puerto en el que queramos levantar la aplicación para poder accederla desde cualquier equipo dentro de la red local de la máquina virtual.
La sintaxis a utilizar sería la siguiente:
“**sudo python manage.py runserver ip:puerto**”

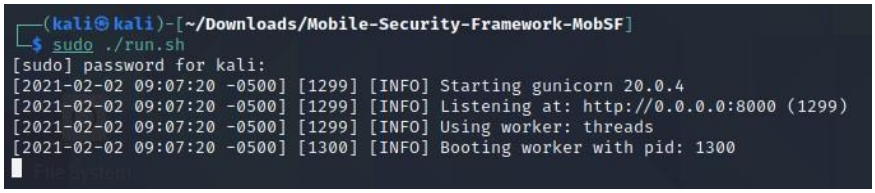


Imagen 28 **Ejecución MOBSF**

Una vez hemos levantado el entorno, debemos iniciar una navegador web e introducir la dirección ip con la hemos iniciado el servicio (http://127.0.0.1:8000 para la configuración por defecto):

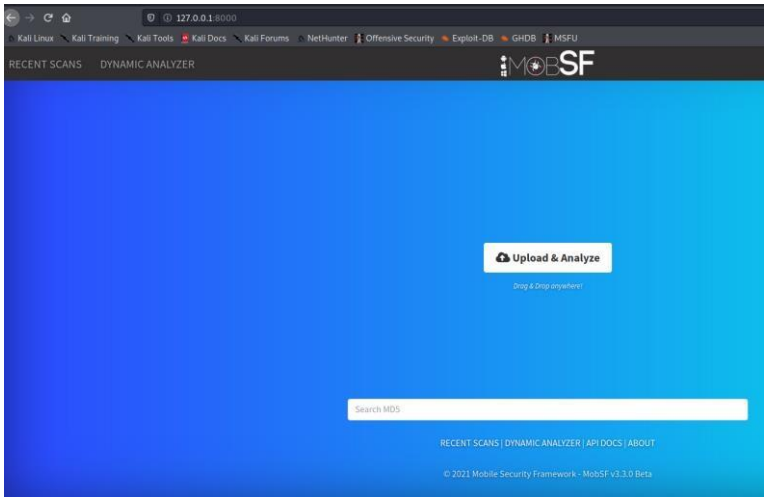


Imagen 29 **Pantalla principal MOBSF**

Tan solo debemos arrastrar el fichero a analizar o bien indicar su ruta. Si disponemos de un hash que queremos verificar si hemos analizado previamente, también podemos buscar por su md5. En nuestro caso utilizaremos la aplicación vulnerable que estamos analizando.

Automáticamente la herramienta realizará el análisis de la aplicación que hayamos seleccionado, mostrando información relativa a la misma:

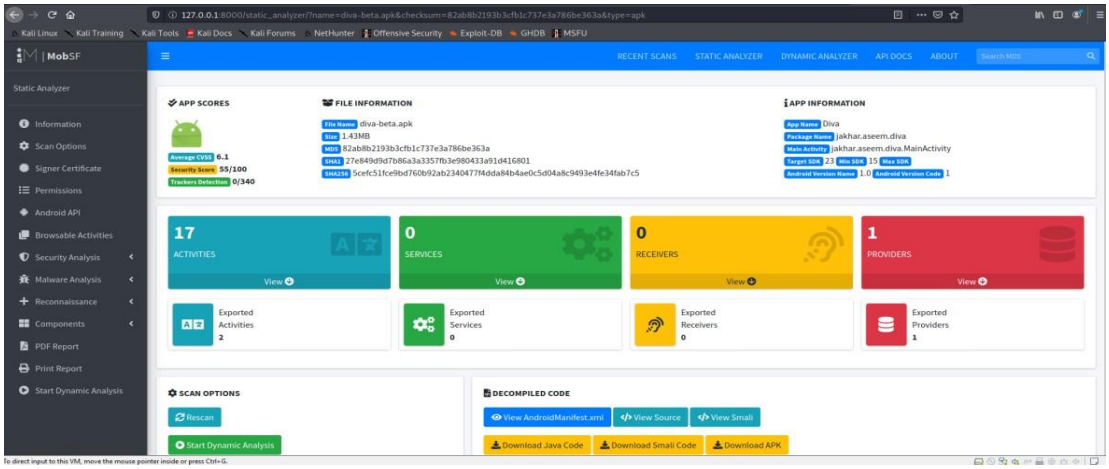


Imagen 30 **Pantalla análisis MOBSF**

Podemos ver como desglosa la aplicación, la hace legible y nos informa del número de activities, content-providers y demás elementos tiene.

En la siguiente captura vemos los **APP Scores**, el cual nos ofrece una valoración cuantificada en cuanto a los niveles de seguridad de la aplicación analizada utilizando diferentes métricas.



Imagen 31 **Pantalla análisis MOBSF APP information**

Vemos también la sección de **Información del archivo**, en la que se encuentra un resumen de las características que podrán permitirnos su posterior identificación: el nombre de la muestra, su tamaño y los hashes (MD5, SHA1, SHA256).

Finalmente, en la sección **Información de la aplicación** encontraremos detalles de la aplicación obtenidos en su mayoría del fichero Android Manifest, entre otros: nombre del paquete, nombre de clase de la actividad principal a ser lanzada por el *laucher* y atributos referentes a los requisitos de la plataforma.

Seguidamente vemos varios widgets que ofrecen información sobre los **Posibles elementos vulnerables**. Encontraremos en la pantalla cuatro recuadros que nos resumen la información referente a las actividades, servicios, receptores de intentos y proveedores de contenidos, indicando cuántos de ellos son exportados. La identificación de estos cuatro elementos es un paso rutinario en cualquier proceso de análisis de malware o pentesting de aplicaciones, ya que nos permitirá no solo saber cómo se comporta la aplicación, sino también descubrir posibles puntos de explotación.

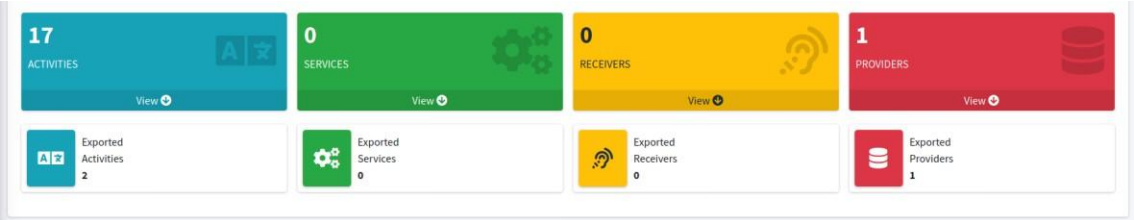


Imagen 33 **MOBSF Componentes aplicación Andorid**

En el análisis de nuestra aplicación vemos como la aplicación tiene 17 activities de las cuales exporta dos. Si pulsamos sobre la flecha pivotamos para ver la información.



Imagen 34 **Actividades APK análisis MOBSF**

Recordemos que las activities están relacionadas principalmente con la interfaz del usuario u las actividades que se lanzan por cada elemento. Podremos hacer eso con cada elemento que nos aporte información como en este caso los Content Providers.



Imagen 35 **Proveedores APK análisis MOBSF**

En la Siguiente imagen vemos las opciones de Análisis del código decompilado. Nos permite acceder a un listado de las clases y **ver su código fuente tanto en formato java como en smali**, y también al archivo manifiesto. Además, encontramos dos opciones: una para escanear nuevamente la muestra y otra para iniciar su análisis dinámico. Podremos de esta forma sin salir del framework ni utilizar otras herramientas poder ver el código fuente y los ficheros de forma visual y sencilla.

Para ilustrar la funcionalidad veamos con un ejemplo como resolver el reto 2 de la aplicación DIVA que estamos analizando. Se trata de una entrada de datos donde introduciendo la key correcta indica un mensaje de acceso correcto y sino mostrará un error.

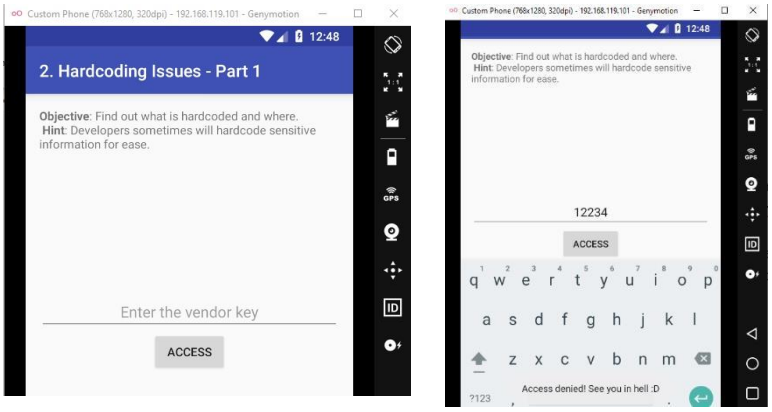


Imagen 36 **DIVA ejercicio 2**

Si analizamos el código fuente, en Mobsf con la opción "View Source", podremos buscar la palabra Key, para tratar de buscar clases en las que se esté utilizando esa palabra.
Examinaremos las clases java que están coloreadas, pq quiere decir que contienen la palabra key, hasta que vemos cual es el fichero que contiene harcodeada la key correcta.

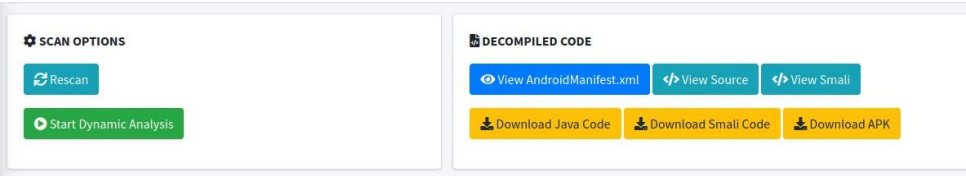


Imagen 37 **decompilación y escaneos APK MOBSF**

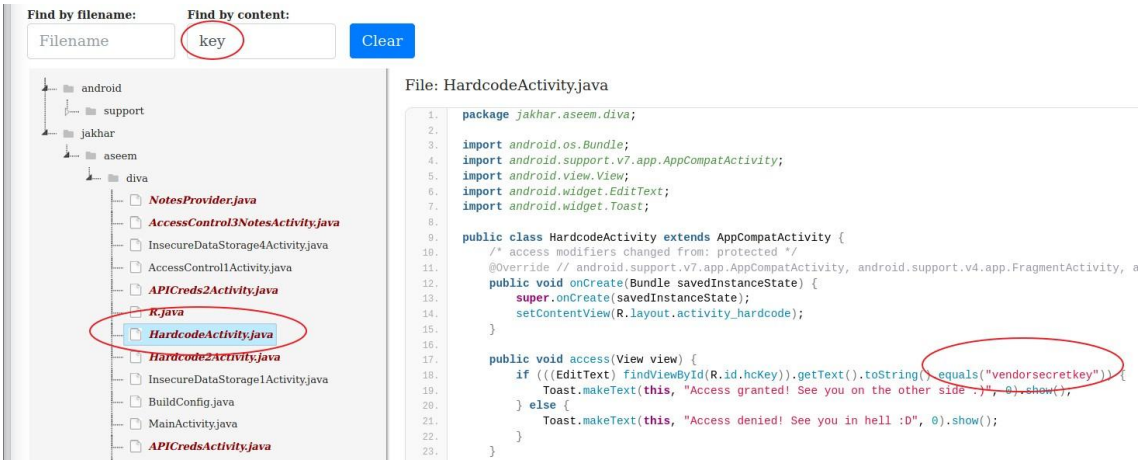


Imagen 38 **APK análisis de código MOBSF**

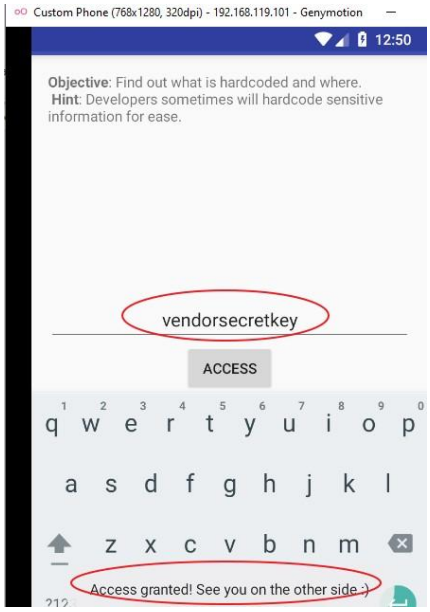


Imagen 39 **DIVA**

Obviamente se trata de un ejercicio muy sencillo, pero nos ayudará a entender el proceso de análisis.

Analizaremos ahora otra sección de código para ver otro ejemplo interesante de análisis estático ya que implica **la obtención de una base de datos** de SQLite muy utilizada en este tipo de plataforma.



Imagen 40 **APK análisis de código MOBSF**

El el código anterior vemos el nombre de una base de datos. En la siguiente captura de pantalla vemos con desde una Shell con adb podemos navegar hasta la ruta donde se encuentran las bases de datos de la aplicación.

```
vbox86p:/data/data # cd jakhar.aseem.diva/databases/
vbox86p:/data/data/jakhar.aseem.diva/databases # ls
divanotes.db divanotes.db-journal sqli sqli-journal
vbox86p:/data/data/jakhar.aseem.diva/databases #
```

Imagen 41 **ADB directorios**

Finalmente, con el comando pull de adb nos podemos traer el archivo para examinarlo posteriormente con cualquier cliente de SQLite y ver su contenido.

```
C:\Windows\system32>adb pull /data/data/jakhar.aseem.diva/databases/divanotes.db
/data/data/jakhar.aseem.diva/databases/divanotes.db: 1 file pulled, 0 skipped. 0.8 MB/s (20480 bytes in 0.024s)
```

Imagen 42 **ADB pull**

Prosigamos con la siguiente opción del menú, veremos la **Información del certificado.** Del análisis del certificado de un APK se pueden obtener datos muy interesantes en cuanto a quién ha desarrollado la aplicación y qué otras aplicaciones maliciosas se han encontrado con el mismo certificado, pudiendo utilizar su identificador para realizar búsquedas en plataformas como Koodous o VirusTotal. Adicionalmente nos ofrece información de contexto. En nuestro caso concreo indica el uso de la firma en versión V1 lo hace vulnerable a Janus, para Android < 7.0. Adicionalmente nos indica que el certificado esta en modo debug.

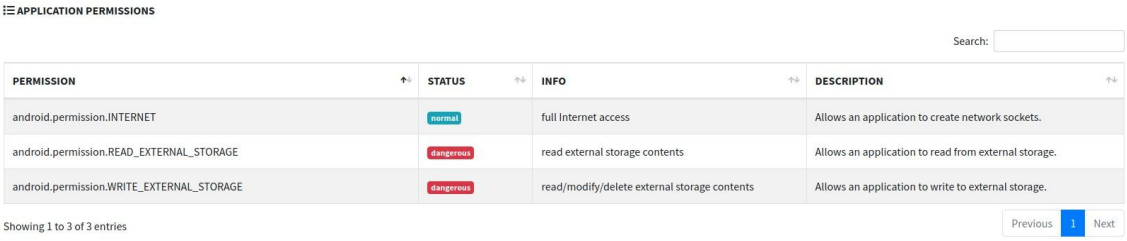
SIGNER CERTIFICATE

```
APK is signed
v1 signature: True
v2 signature: False
v3 signature: False
Found 1 unique certificates
Subject: C=US, O=Android, CN=Android Debug
Signature Algorithm: rsassa_pkcs1v15
Valid From: 2015-11-02 08:32:11+00:00
Valid To: 2045-10-25 08:32:11+00:00
Issuer: C=US, O=Android, CN=Android Debug
Serial Number: 0x218330df
Hash Algorithm: sha256
md5: d620162ac34ee974d7fd3a1862e7e4df
sha1: ae4ead5aeaba4e9e4fc928e7c7f7fd459f008031
sha256: 35d7f7ad35dfb826b70fa4b73187ed478540e32c8b8c5653b86568029fcd5840
sha512: e936169585893a7a248e393ddb296b2101432de5b07c3d7e2bdc8070dc0b58277b46e443eff3730b4f5a25b61f78c9078f54cc325cb86c17160b5bae13148d1e
```

STATUS	DESCRIPTION
bad	Application is signed with v1 signature scheme, making it vulnerable to Janus vulnerability on Android <7.0
bad	Application signed with a debug certificate. Production application must not be shipped with a debug certificate.
secure	Application is signed with a code signing certificate

Imagen 43 **Certificado APK MOBSF**

En la siguiente sección podremos observar una **lista de los permisos** declarados en el manifiesto de la aplicación, conjuntamente a una descripción del mismo y una categorización según la peligrosidad que puede representar para el sistema al acceder a información o funcionalidad sensible.



APPLICATION PERMISSIONS

Search:

PERMISSION	STATUS	INFO	DESCRIPTION
android.permission.INTERNET	normal	full Internet access	Allows an application to create network sockets.
android.permission.READ_EXTERNAL_STORAGE	dangerous	read external storage contents	Allows an application to read from external storage.
android.permission.WRITE_EXTERNAL_STORAGE	dangerous	read/modify/delete external storage contents	Allows an application to write to external storage.

Showing 1 to 3 of 3 entries

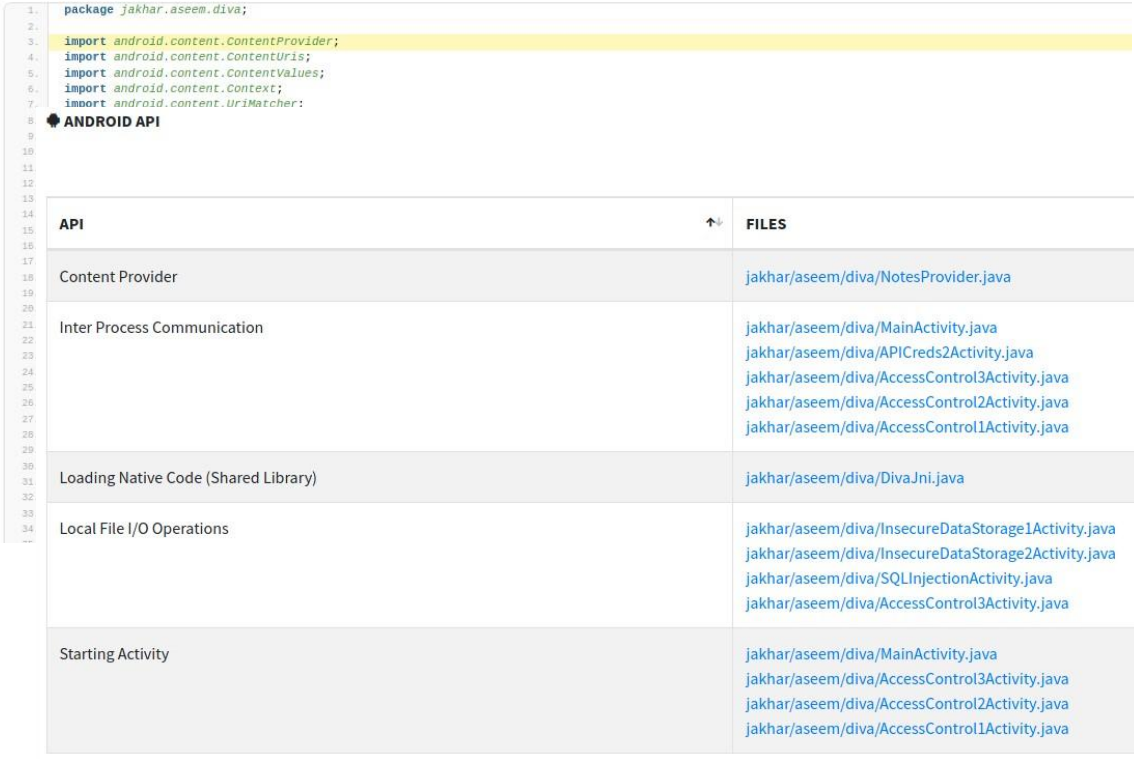
Previous 1 Next

Imagen 44 **Permisos aplicación MOBSF**

Del análisis de nuestra aplicación podemos ver que los permisos de lectura y escritura en almacenamiento externo pueden ser peligrosas. La de conexión a internet, en función del contexto de la aplicación puede ser peligrosa, en este caso no se ha considerado así por la herramienta, pero es algo que habríamos de validar. El ejemplo más clásico es: ¿Necesita una app de linterna acceso a internet, o a almacenamiento? La respuesta es casi siempre NO … así que deberemos buscar las clases que los utilizan porque probablemente sean maliciosas.

Ahora veremos la información relativa a la Android API. Resulta muy útil para un analista, ya que permite identificar rápidamente qué funcionalidades de la API del sistema son accedidas por cada clase de la aplicación. De este modo, es muy sencillo identificar qué función realiza cada clase y podremos concentrarnos en aquello que realmente nos interese. Es decir, nos hablará de su funcionalidad basada

NotesProvider.java
en los métodos invocados.



```
package jakhar.aseem.diva;

import android.content.ContentProvider;
import android.content.ContentUris;
import android.content.ContentValues;
import android.content.Context;
import android.content.UriMatcher;
```

ANDROID API

API	FILES
Content Provider	jakhar/aseem/diva/NotesProvider.java
Inter Process Communication	jakhar/aseem/diva/MainActivity.java jakhar/aseem/diva/APICreds2Activity.java jakhar/aseem/diva/AccessControl3Activity.java jakhar/aseem/diva/AccessControl2Activity.java jakhar/aseem/diva/AccessControl1Activity.java
Loading Native Code (Shared Library)	jakhar/aseem/diva/DivaJni.java
Local File I/O Operations	jakhar/aseem/diva/InsecureDataStorage1Activity.java jakhar/aseem/diva/InsecureDataStorage2Activity.java jakhar/aseem/diva/SQLInjectionActivity.java jakhar/aseem/diva/AccessControl3Activity.java
Starting Activity	jakhar/aseem/diva/MainActivity.java jakhar/aseem/diva/AccessControl3Activity.java jakhar/aseem/diva/AccessControl2Activity.java jakhar/aseem/diva/AccessControl1Activity.java

Imagen 45 **APK API MOBSF**

En nuestro análisis podemos ver como las API relativas a Local File I/O Operations, Loading Native Code (Shared Library) son utilizadas en diferentes partes de la aplicación. Podremos pivotar sobre ellas para analizarlas en detalle. Por ejemplo, del análosos de NotesProvider.java vemos lo siguiente:

NotesProvider.java

```
package jakhar.aseem.diva;

import android.content.ContentProvider;
import android.content.ContentUris;
import android.content.ContentValues;
import android.content.Context;
import android.content.UriMatcher;
import android.database.Cursor;
import android.database.SQLException;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
import android.database.sqlite.SQLiteQueryBuilder;
import android.net.Uri;
import android.text.TextUtils;

public class NotesProvider extends ContentProvider {
    static final String AUTHORITY = "jakhar.aseem.diva.provider.notesprovider";
    static final Uri CONTENT_URI = Uri.parse("content://jakhar.aseem.diva.provider.notesprovider/notes");
    static final String CREATE_TBL_QRY = " CREATE TABLE notes (_id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT NOT NULL, note TEXT NOT NULL);";
    static final String C_ID = "_id";
    static final String C_NOTE = "note";
    static final String C_TITLE = "title";
    static final String DBNAME = "divanotes.db";
    static final int DBVERSION = 1;
    static final String DROP_TBL_QRY = "DROP TABLE IF EXISTS notes";
    static final int PATH_ID = 2;
    static final int PATH_TABLE = 1;
    static final String TABLE = "notes";
    static final UriMatcher urimatcher = new UriMatcher(-1);
    SQLiteDatabase mDB;

    static {
        urimatcher.addURI(AUTHORITY, TABLE, 1);
        urimatcher.addURI(AUTHORITY, "notes/#", 2);
```

Imagen 46 **Análisis de código MOBSF**

Como vemos en la imagen anterior, podemos saber que la base de datos se llama divanotes.db, también que la tabla se llama notes, etc.. es decir, obtenemos información para explotar posibles vulnerabilidades de SQLI por ejemplo.

Deberemos revisar todos los .java de interés para encontrar información sensible, posibles vulnerabilidades y entender grosso modo la estructura de su programación. Además de las secciones antes discutidas, podremos encontrar otras categorías con detalles de elementos a ser considerados en cualquier análisis. Por ejemplo, podremos ver una sección donde se especifica con detalle cuáles son las actividades, servicios, broadcast receivers y content providers especificados en la aplicación, o podremos acceder a un listado de las strings encontradas dentro del código fuente. A continuación, examinaremos veremos que se nos ofrecen una serie de análisis específicos de seguridad que la herramienta realizará de forma automática.

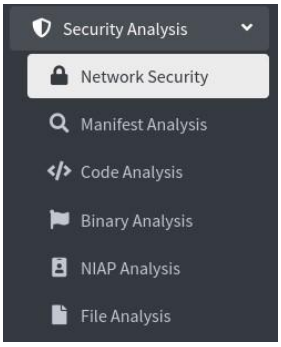
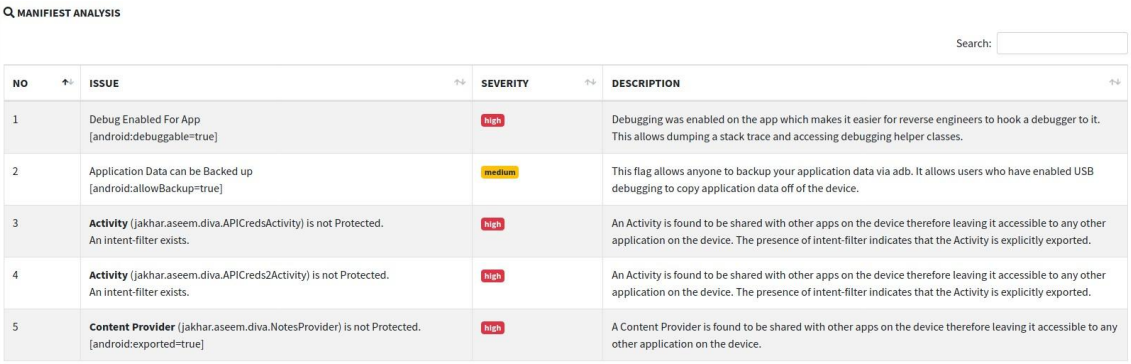


Imagen 47 **Análisis de seguridad MOBSF**

En la siguiente captura vemos como directamente nos ofrece un análisis de este fichero, ofreciendo no sólo la identificación del problema, sino que también indica severidad y una descripción.



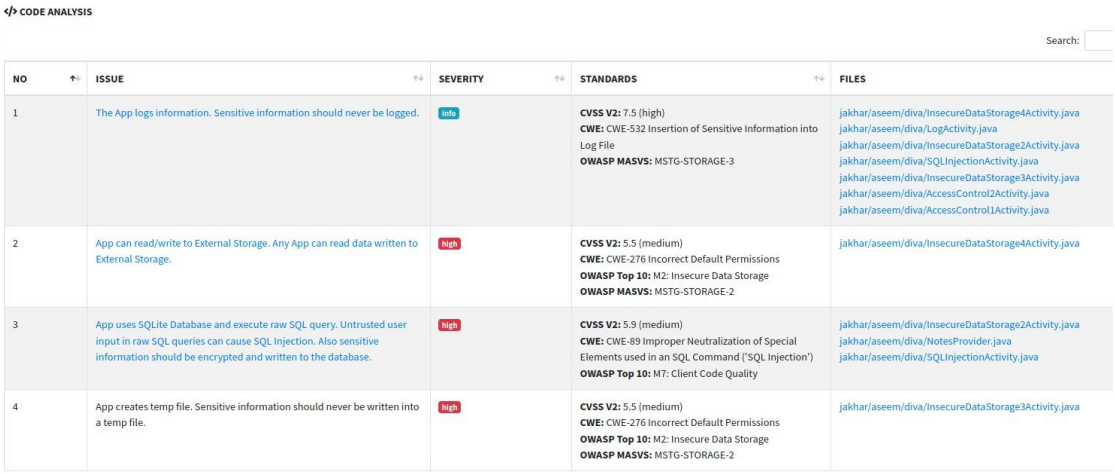
MANIFIEST ANALYSIS

Search:

NO	ISSUE	SEVERITY	DESCRIPTION
1	Debug Enabled For App [android:debuggable=true]	high	Debugging was enabled on the app which makes it easier for reverse engineers to hook a debugger to it. This allows dumping a stack trace and accessing debugging helper classes.
2	Application Data can be Backed up [android:allowBackup=true]	medium	This flag allows anyone to backup your application data via adb. It allows users who have enabled USB debugging to copy application data off of the device.
3	**Activity** (jakhar.aseem.diva.APICredsActivity) is not Protected. An intent-filter exists.	high	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Activity is explicitly exported.
4	**Activity** (jakhar.aseem.diva.APICreds2Activity) is not Protected. An intent-filter exists.	high	An Activity is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device. The presence of intent-filter indicates that the Activity is explicitly exported.
5	**Content Provider** (jakhar.aseem.diva.NotesProvider) is not Protected. [android:exported=true]	high	A Content Provider is found to be shared with other apps on the device therefore leaving it accessible to any other application on the device.

Imagen 48 **Análisis Manifiest MOBSF**

Del resultado del análisis vemos problemas de seguridad relativos a una mayor facilidad de análisis al encontrarse el flag de bebug activo, permitiendo al analista utilizar un debuguer. Esto no es siempre así, será interesante ver más adelante como resolver este problema para poder debuguear dinámicamente la aplicación.
Como veremos en las dos imágenes siguientes, realizará un análisis similar al anterior sobre el código fuente, las librerías compartidas.

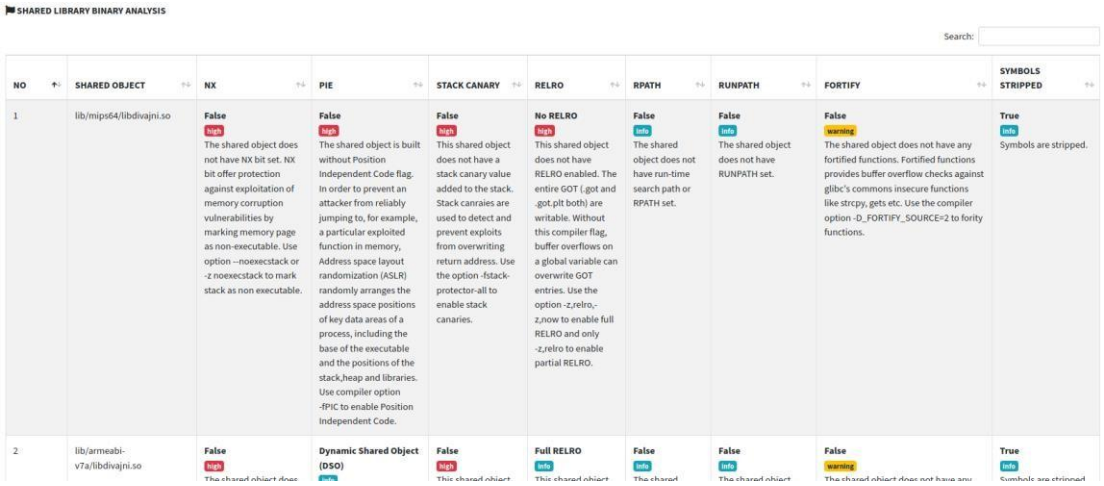


</> CODE ANALYSIS

Search:

NO	ISSUE	SEVERITY	STANDARDS	FILES
1	The App logs information. Sensitive information should never be logged.	info	**CVSS V2:** 7.5 (high) **CWE:** CWE-532 Insertion of Sensitive Information into Log File **OWASP MASVS:** MSTG-STORAGE-3	jakhar/aseem/diva/InsecureDataStorage4Activity.java jakhar/aseem/diva/LogActivity.java jakhar/aseem/diva/InsecureDataStorage2Activity.java jakhar/aseem/diva/SQLInjectionActivity.java jakhar/aseem/diva/InsecureDataStorage3Activity.java jakhar/aseem/diva/AccessControl2Activity.java jakhar/aseem/diva/AccessControl1Activity.java
2	App can read/write to External Storage. Any App can read data written to External Storage.	high	**CVSS V2:** 5.5 (medium) **CWE:** CWE-276 Incorrect Default Permissions **OWASP Top 10:** M2: Insecure Data Storage **OWASP MASVS:** MSTG-STORAGE-2	jakhar/aseem/diva/InsecureDataStorage4Activity.java
3	App uses SQLite Database and execute raw SQL query. Untrusted user input in raw SQL queries can cause SQL Injection. Also sensitive information should be encrypted and written to the database.	high	**CVSS V2:** 5.9 (medium) **CWE:** CWE-89 Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') **OWASP Top 10:** M7: Client Code Quality	jakhar/aseem/diva/InsecureDataStorage2Activity.java jakhar/aseem/diva/NotesProvider.java jakhar/aseem/diva/SQLInjectionActivity.java
4	App creates temp file. Sensitive information should never be written into a temp file.	high	**CVSS V2:** 5.5 (medium) **CWE:** CWE-276 Incorrect Default Permissions **OWASP Top 10:** M2: Insecure Data Storage **OWASP MASVS:** MSTG-STORAGE-2	jakhar/aseem/diva/InsecureDataStorage3Activity.java

Imagen 49 **Análisis de código MOBSF**



SHARED LIBRARY BINARY ANALYSIS

Search:

NO	SHARED OBJECT	NX	PIE	STACK CANARY	RELRO	RPATH	RUNPATH	FORTIFY	SYMBOLS STRIPPED
1	lib/mips64/libdivajni.so	**False** high The shared object does not have NX bit set. NX bit offer protection against exploitation of memory corruption vulnerabilities by marking memory page as non-executable. Use option --noexecstack or -z noexecstack to mark stack as non executable.	**False** high The shared object is built without Position Independent Code flag. In order to prevent an attacker from reliably jumping to, for example, a particular exploited function in memory, Address space layout randomization (ASLR) randomly arranges the address space positions of key data areas of a process, including the base of the executable and the positions of the stack,heap and libraries. Use compiler option -fPIC to enable Position Independent Code.	**False** high This shared object does not have a stack canary value added to the stack. Stack canraies are used to detect and prevent exploits from overwriting return address. Use the option -fstack-protector-all to enable stack canaries.	**No RELRO** high This shared object does not have RELRO enabled. The entire GOT (.got and .got.plt both) are writable. Without this compiler flag, buffer overflows on a global variable can overwrite GOT entries. Use the option -z,relro,-z,now to enable full RELRO and only -z,relro to enable partial RELRO.	**False** info The shared object does not have run-time search path or RPATH set.	**False** info The shared object does not have RUNPATH set.	**False** warning The shared object does not have any fortified functions. Fortified functions provides buffer overflow checks against glibc's commons insecure functions like strcpy, gets etc. Use the compiler option -D_FORTIFY_SOURCE=2 to fority functions.	**True** info Symbols are stripped.
2	lib/armeabi-v7a/libdivajni.so	**False** high	**Dynamic Shared Object (DSO)** info	**False** high	**Full RELRO** info	**False** info	**False** info	**False** warning	**True** info

Imagen 50 **Análisis de librerías compartidas MOBSF**

En el siguiente apartado elabora un informe basado en NIAP. La National Information Assurance Partnership (NIAP) es responsable de la implementación de los Criterios Comunes en los Estados Unidos, incluyendo la gestión del organismo de validación NIAP Common Criteria Evaluation and Validation Scheme (CCEVS). El NIAP gestiona un programa nacional para el desarrollo de perfiles de protección, metodologías de evaluación y políticas que garanticen unos requisitos alcanzables, repetibles y comprobables.

NIAP ANALYSIS v1.3

Search:

NO	IDENTIFIER	REQUIREMENT	FEATURE	DESCRIPTION
1	FCS_STO_EXT.1.1	Security Functional Requirements	Storage of Credentials	The application does not store any credentials to non-volatile memory.
2	FCS_CKM_EXT.1.1	Security Functional Requirements	Cryptographic Key Generation Services	The application generate no asymmetric cryptographic keys.
3	FDP_DEC_EXT.1.1	Security Functional Requirements	Access to Platform Resources	The application has access to ['network connectivity'].
4	FDP_DEC_EXT.1.2	Security Functional Requirements	Access to Platform Resources	The application has access to no sensitive information repositories.
5	FDP_NET_EXT.1.1	Security Functional Requirements	Network Communications	The application has user/application initiated network communications.
6	FDP_DAR_EXT.1.1	Security Functional Requirements	Encryption Of Sensitive Application Data	The application implement functionality to encrypt sensitive data in non-volatile memory.
7	FMT_MEC_EXT.1.1	Security Functional Requirements	Supported Configuration Mechanism	The application invoke the mechanisms recommended by the platform vendor for storing and setting configuration options.
8	FTP_DIT_EXT.1.1	Security Functional Requirements	Protection of Data in Transit	The application does not encrypt any data in traffic or does not transmit any data between itself and another trusted IT product.

Imagen 51 **Análisis NIAP MOBSF**

Para finalizar esta sección veremos detalles relativos al análisis del fichero clases.dex

APKiD ANALYSIS

DEX	DETECTIONS
classes.dex	

FINDINGS	DETAILS
Compiler	dx (possible dexmerge)
Manipulator Found	dexmerge

Showing 1 to 2 of 2 entries

Imagen 52 **Análisis APKid MOBSF**

En ellas podemos encontrar detalles interesantes, como en este caso en el que vemos manipulación del fichero evidenciada por el uso de dexmerge. Veamos en detalle que significa esta detección.

Las aplicaciones de Android son mucho más fáciles de modificar que las de los sistemas operativos de escritorio tradicionales como Windows o Linux, y principalmente hay una sola manera de modificar las aplicaciones de Android después de que se hayan compilado. Mediante la técnica denominada **inyección Smali**, la cual consiste en desensamblar el código de la aplicación original con la herramienta Baksmali, añadir el código malicioso y ensamblarlo con Smali. Como resultado, debido a un proceso de compilación no standar, se dejan ciertas marcas en el archivo dex que apuntan a dexlib, una biblioteca utilizada por la herramienta Smali para ensamblar archivos dex.
Es decir, cualquier aplicación a la que se le haya inyectado código malicioso o que haya sido crackeada o pirateada probablemente habrá sido desensamblada y recompilada por dexlib.

Además, hay muy pocas razones por las que un desarrollador con acceso al código fuente utilizaría dexlib. Por lo tanto, si sabes que una aplicación ha sido modificada por dexlib, probablemente te interese si te preocupa el malware o la piratería de aplicaciones. Aquí es donde entra APKiD.

Además de detectar empaquetadores, ofuscadores y otras cosas raras, también puede identificar si una aplicación fue compilada por los compiladores estándar de Android o por dexlib ya que aunuq se use apktool o use Smali, estaremos por debajo teniendo dexlib.

En nuestro ejemplo, se han encontrado rastros de compiladores dx/dexmerge, lo que significa que importaron la fuente original del código en un IDE de Android y lo compilaron con sus propias modificaciones.

La principal forma de identificar dx y dexmerge es mirando el orden de los tipos de mapas en el archivo DEX.

En la siguiente sección nos ofrece diferentes herramientas que busacan tipos de datos interesantes como urls sobre las que ofrecerán datos, cadenas de texto o password entre otras funciones.

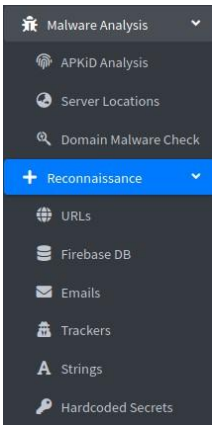


Imagen 53 **Análisis de malware MOBSF**

Como vemos a continuación hay Strings de interés y un parámetro llamado pkey que parece de interés.

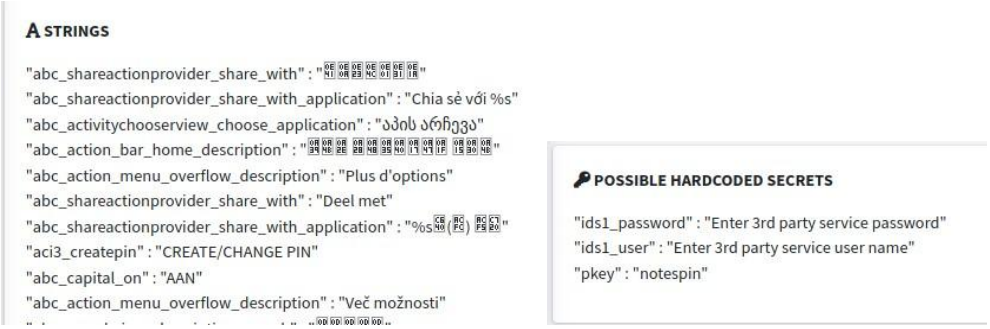


Imagen 54 **Cadenas de tecto MOBSF**

Al igual que otras aplicaciones diseñadas para el mismo cometido, MobSF nos permite generar y consultar un reporte con todos los datos relevantes de la aplicación analizada:

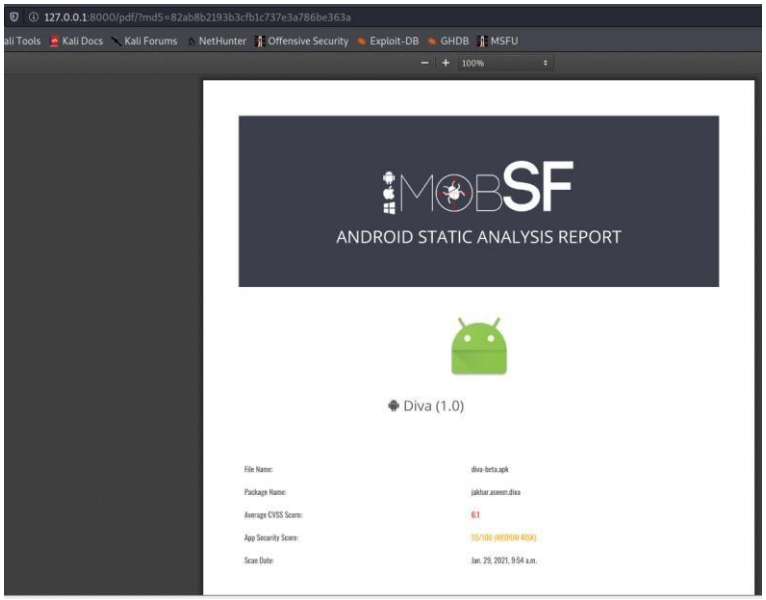


Imagen 55 **Reporte MOBSF**

10.5. ANÁLISIS ESTÁTICO EN IOS

Sabemos que se pueden utilizar los lenguajes Objective-C y Swift para cerar aplicación para entornos IoS. También sabemos que ambos lenguajes son fundamentalmente diferentes, por lo tanto, el lenguaje de programación en el que está escrita la aplicación afecta a las posibilidades de realizar ingeniería inversa. Por ejemplo, Objective-C permite cambiar las invocaciones de los métodos en tiempo de ejecución. Esto facilita el enganche a otras funciones de la aplicación (una técnica muy utilizada por Cycript y otras herramientas de ingeniería inversa). Esta uso de los métodos no se implementa de la misma manera en Swift, y la diferencia hace que la técnica sea más difícil de ejecutar con Swift que con Objective-C.

En iOS, todo el código de la aplicación (tanto Swift como Objective-C) se compila a código máquina (por ejemplo, ARM). Por lo tanto, para analizar las aplicaciones de iOS se necesita un desensamblador.

En esta sección, el término "binario de la aplicación" se refiere al archivo Macho-O del paquete de la aplicación que contiene el código compilado, y no debe confundirse con el paquete de la aplicación, el archivo IPA.
Para el análisis de ficheros .ipa seguiremos la siguiente estructura de trabajo:

Extracción de los archivos de la aplicación: Extraer el contenido del .ipa es tan sencillo como cambiar su extensión a .zip y descomprimirlo. Después de descomprimir el contenido veremos una carpeta llamada Payload y dentro encontrarás el paquete de aplicación. En una aplicación

descargada de la App Store encontrarás 2 archivos más, un archivo iTunesArtwork que es el icono de la aplicación y un archivo iTunesMetadata.plist que contiene información como el nombre y el ID del desarrollador, el identificador del bundle, los derechos de autor, el nombre de la aplicación, correo electrónico del comprador y fecha de compra entre otros datos.

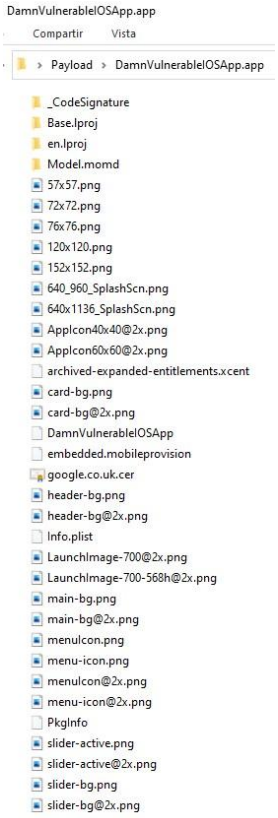


Imagen 56 **Estructura de ficheros IPA**

 Analizar los archivos incrustados: En iOS podemos encontrar archivos de configuración, archivos de datos de ejemplo, archivos de conexión de base
de datos o claves privadas incrustadas para las conexiones SSH entre otras posibles vulnerabilidades.

Los dos archivos de configuración más comunes en iOS son los .plist y los .json. Es interesante comenzar leyendo todos los archivos que puedas encontrar con estas extensiones y mira si hay alguna información que no debería estar ahí. Un archivo muy importante es el Info.plist que podemos encontrar en el directorio raíz de una aplicación iOS.



Imagen 57 **Ficheros de configuración .plist en IoS**

Este archivo contiene muchos datos de configuración, como por ejemplo si la aplicación permite una configuración TLS débil en algunos dominios (busque la clave NSAppTransportSecurity), o si la aplicación acepta URLs de Scheme personalizadas (clave CFBundleURLTypes).

Los modelos CoreData compilados (.mom, .momd) pueden ser descompilados en archivos xcdatamodel utilizando la herramienta momdec. Estos archivos pueden ser inspeccionados posteriormente en Xcode.

- **Analizar los frameworks de terceros:** Casi todas las aplicaciones de iOS utilizan al menos un framework de terceros. Este es un detalle muy importante porque esto aumenta la superficie de ataque, y la mayoría de las veces los desarrolladores se olvidan de actualizar sus dependencias, cuanto más grande es la lista de dependencias, más difícil es hacer un seguimiento de las versiones actualizadas. Esto significa que mientras una aplicación "siga funcionando" no hay ningún incentivo para actualizar estos frameworks de terceros. Esto deja a los usuarios con un código obsoleto y potencialmente vulnerable en sus dispositivos. Todos los frameworks de terceros dentro de un paquete de iOS se encuentran en una carpeta llamada Frameworks. Abriremos esa carpeta y prestaremos atención a las versiones de los frameworks.
    - o Consejo 1: La versión del framework se revela en su archivo info.plist. o Consejo 2: Busca en Google esas versiones de frameworks y busca vulnerabilidades conocidas.Veremos herramientas comop MOBSF que harán esta tarea de forma automática por nosotros.

- **Volcado de las clases de la aplicación:** Una parte esencial del análisis estático de cualquier aplicación es recopilar información sobre qué métodos y clases contiene la aplicación. Este paso proporciona información muy importante porque, como muchos desarrolladores saben, declarar métodos muy descriptivos ayuda al desarrollo de buenos productos. Así, los nombres de algunos de los métodos darán una idea de cuáles son las características de la aplicación. Usaremos herramientas como class-dump-z para volcar las clases y métodos de la aplicación.
- **Desensamblado y decompilado del binario:** Después del trabajo inicial de reconocimiento, comenzaremos a entender el comportamiento real de los métodos de la aplicación. Después de buscar a través de las clases y métodos en el volcado de clases de la fase previa, pondremos foco en las clases más interesantes, dado que el volumen de las aplicaciones suele hacer imprescindible priorizar el trabajo y centrarse en las clases más interesantes.

Vulnerabilidades más comunes IoS: **Enlace**

En este caso la aplicación sobre la que vamos a realizar el análisis es: **Descargar**

Para los que dispongan de un dispositivo IoS rooteado, entornos adecuados y quieran analizar en profundidad todas las vulnerabilidades que la aplicación que utilizaremos nos ofrece, recomiendo seguir los tutoriales de la web. En sí sólo este capítulo necesitaría un módulo entero, por lo que nos hemos centrado en todo el conocimiento inicial necesario.

Tutoriales Damm Vulnerable APP: **Enlace**

10.5.1. TÉCNICAS Y HERRAMIENTAS

Todas las herramientas orientadas a analizar aplicaciones iOS buscan el mismo cometido: extraer el mayor número posible de información para su posterior análisis. Con el objetivo de reaprovechar software para analizar ambos entornos y reducir el número de herramientas a instalar, pondremos foco en MobSF, ya que también permite análisis estático en IoS y ofrece una multitud de funcionalidades al respecto en una única solución.

10.5.1.1. MOBSF

Al igual que en un entorno de Android, esta herramienta nos permite analizar, de un plumazo y desde un entorno visual, información sobre la aplicación que queremos analizar (en este caso concreto utilizaremos el fichero ".ipa" correspondiente a la aplicación DamnVulnerableIOSApp).

- Una vez seleccionada la aplicación, podemos ver como la herramienta nos facilita toda la información posible sobre la misma, clasificada en diferentes apartados.

Oservamos como nos muestra información sobre la aplicación del mismo modo que vimos en Android:

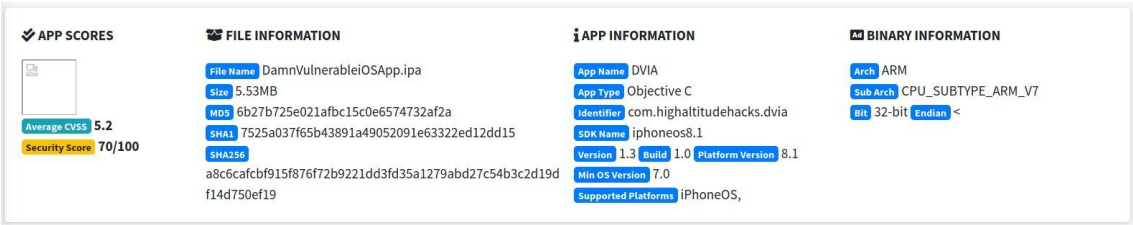


Imagen 58 **Análisis aplicación IPA en MOBSF**

En las siguiente captura vemos cómo es posible acceder y visualizar el fichero info.plist, el cual ya hemos mencionado su importancia.



Imagen 59 **Opciones de escaneo en MOBSF**

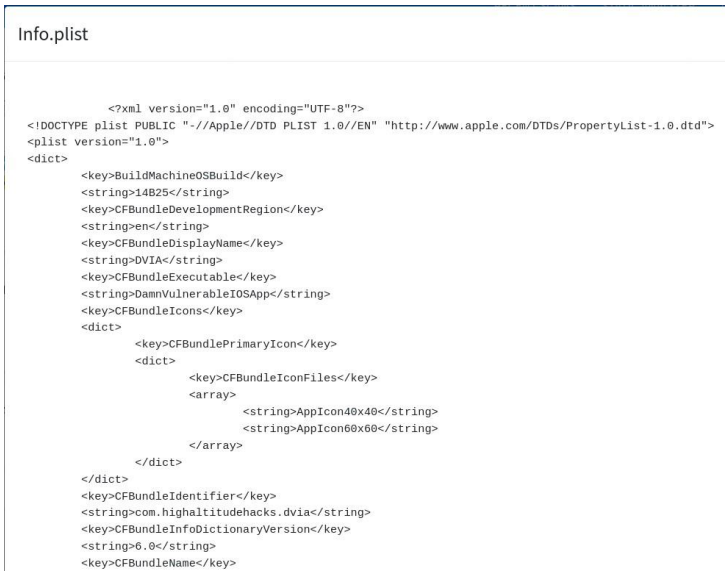


Imagen 60 **Análisis .plist en MOBSF**

Leyendo el archivo plist, podemos reunir alguna información básica sobre la aplicación como

- Información de la versión: La versión actual de la aplicación. Se almacena en la clave ''.
- Nombre de la aplicación: El nombre de la aplicación tal y como se muestra en el iPhone. Se almacena en la clave 'CFBundleDisplayName'.
- Nombre del directorio .app de la aplicación: El directorio que contiene el Bundle Container de una aplicación iOS. Se almacena en la clave 'CFBUndleName'.
- Identificador de paquete de la aplicación: El identificador que identifica de forma única una aplicación de otras aplicaciones en el dispositivo. Se almacena en la clave 'CFBundleIdentifier'.
- Nombre binario ejecutable: El archivo ejecutable que contiene el código máquina de una aplicación. Se almacena en la clave "CFBundleExecutable".
- Versión de iOS y dispositivos compatibles: La especificación de la versión de iOS y los modelos de dispositivos compatibles con la aplicación. Los dispositivos se especifican mediante la clave 'UISupportedDevices' y la versión mínima de iOS mediante 'MinimumOSVersion'.
- Nombres de archivos de imagen de iconos y de inicio: El nombre de los archivos de imagen que se utilizarán como iconos e imágenes de pantalla de inicio para los diferentes modelos de iPhones. Se almacena en la clave 'CFBundleIcons'.

- Permisos de la aplicación: La descripción dada por la aplicación para el uso de los permisos sensibles del usuario. Se especifica en las claves "NS<requisito>Descripción de uso".

Seguridad de transporte de la aplicación: La información relativa a las URLs/dominios que la aplicación tiene permitido o no cargar. Se especifica en la clave "NSAppTransportSecurity".

Una vez que hemos obtenido toda la información que deseábamos del archivo Info.plist. Desde el punto de vista de la seguridad, tenemos que comprobar los siguientes controles de este archivo:

- **Permisos innecesarios solicitados por la aplicación:** Una aplicación puede solicitar permisos que no son esenciales para la funcionalidad de la aplicación y pueden ser utilizados maliciosamente por cualquier otra aplicación o servicio en el dispositivo. La habilitación de permisos puede hacer que el usuario haga algunas cosas no deseadas, como compartir la ubicación con terceros remotos, o puede utilizarse para actividades de ingeniería social mediante el uso de componentes del dispositivo como el micrófono, la cámara, etc.

- **Seguridad de transporte de aplicaciones mal configurada:** Una aplicación puede estar mal configurada para permitir la carga de URLs arbitrarias en la aplicación, lo que puede causar problemas de seguridad provocando un comportamiento malicioso en la aplicación.

Tambien podremos ver las **Strings** contenidas en el binario con la siguiente opción, lo cual suele ser de interés por ejemplo para buscar password harcodeadas entre otra información de interés.



Imagen 61 **Strings en MOBSF**

Finalmente vemos la opción de ver **el dumpeo de clases**, **el cual no está accesible al no estar trabajando sobre una plataforma adecuada para que la herramienta funcione**. En el caso de disponer de plataformas adecuadas si es posible ver ese dumpeo de clases:

Class Dump (información referente al tiempo de ejecución de la aplicación, donde se han generado clases, categorías…):

classdump.txt

```
/**
 * This header is generated by class-dump-z 0.2a.
 * class-dump-z is Copyright (C) 2009 by KennyTM~, licensed under GPLv3.
 *
 * Source: (null)
 */

typedef struct _NSZone NSZone;

typedef struct CGPoint {
    float _field1;
    float _field2;
} CGPoint;

typedef struct _NSRange {
    unsigned _field1;
    unsigned _field2;
} NSRange;

typedef struct CGSize {
    float _field1;
    float _field2;
} CGSize;

typedef struct CGRect {
    CGPoint _field1;
    CGSize _field2;
} CGRect;

typedef struct CGAffineTransform {
    float a;
    float b;
    float c;
    float d;
    float tx;
    float ty;
} CGAffineTransform;

typedef struct _RNCryptorKeyDerivationSettings {
    unsigned keySize;
    unsigned saltSize;
    unsigned PBKDFAlgorithm;
    unsigned PRF;
    unsigned rounds;
```

Imagen 62 **Dumpeo de clases en MOBSF**

Veremos a continuación el análisis del propio binario, en el cual observamos alertas de severidad alta al utilizar API con vulnerabilidades conocidas. También vemos como utiliza la función malloc para reserva dinámica de memoria, la cual es insegura por su posibilidad de explotación en buffer overflow.

</> IPA BINARY CODE ANALYSIS

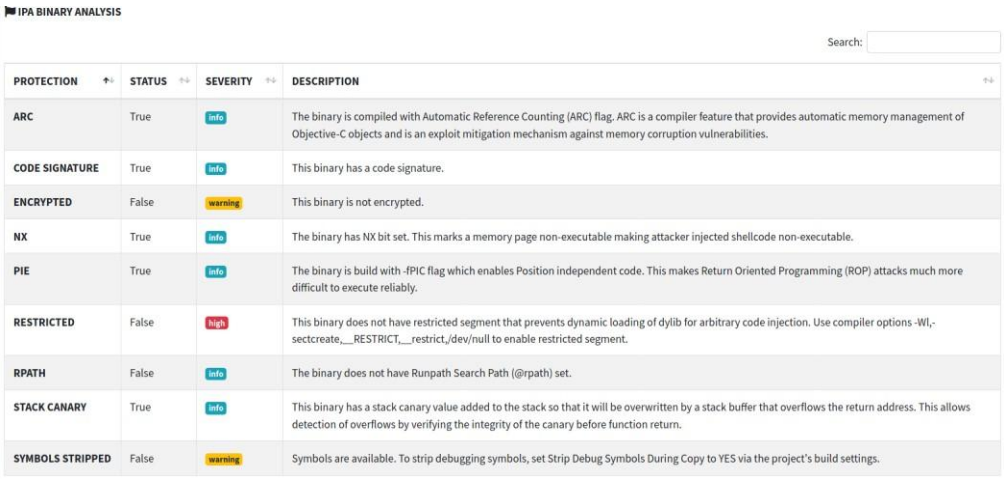
Search:

NO	ISSUE	SEVERITY	STANDARDS	DESCRIPTION
1	Binary makes use of insecure API(s)	high	**CVSS V2:** 6 (medium) **CWE:** CWE-676 - Use of Potentially Dangerous Function **OWASP Top 10:** M7: Client Code Quality **OWASP MASVS:** MSTG-CODE-8	The binary may contain the following insecure API(s) _stat , _sscanf , _memcpy , _strlen .
2	Binary makes use of Logging function	info	**CVSS V2:** 7.5 (high) **CWE:** CWE-532 Insertion of Sensitive Information into Log File **OWASP MASVS:** MSTG-STORAGE-3	The binary may use _NSLog function for logging.
3	Binary makes use of malloc function	high	**CVSS V2:** 2 (low) **CWE:** CWE-789 Uncontrolled Memory Allocation **OWASP Top 10:** M7: Client Code Quality **OWASP MASVS:** MSTG-CODE-8	The binary may use _malloc function instead of calloc.
4	Binary uses WebView Component.	info	**CVSS V2:** 0 (info) **OWASP MASVS:** MSTG-CODE-9	The binary may use UIWebView Component.

Imagen 63 **Análisis de código en MOBSF**

En la siguiente captura vemos que la protección PIE esta activada. Esta relacionada con ASLR (Address Space Layout Randomization). Al igual que en otras arquitecturas, esta técnica dificulta la explotación de alguna vulnerabilidad asignando localizaciones aleatorias de los objetos en memoria. Para usar la protección completa ASLR, y no de forma limitada como lo hacen las aplicaciones iOS por defecto, hay que compilar desde Xcode con el flag PIE activado.

Podemos ver en la imagen siguiente cómo la aplicación ha sido compilada con el flag PIE, por lo tanto, está protegida totalmente con ASLR, y aunque se conocen técnicas para hacer «bypass» a estas protecciones, nunca viene de más poner obstáculos a posibles atacantes.



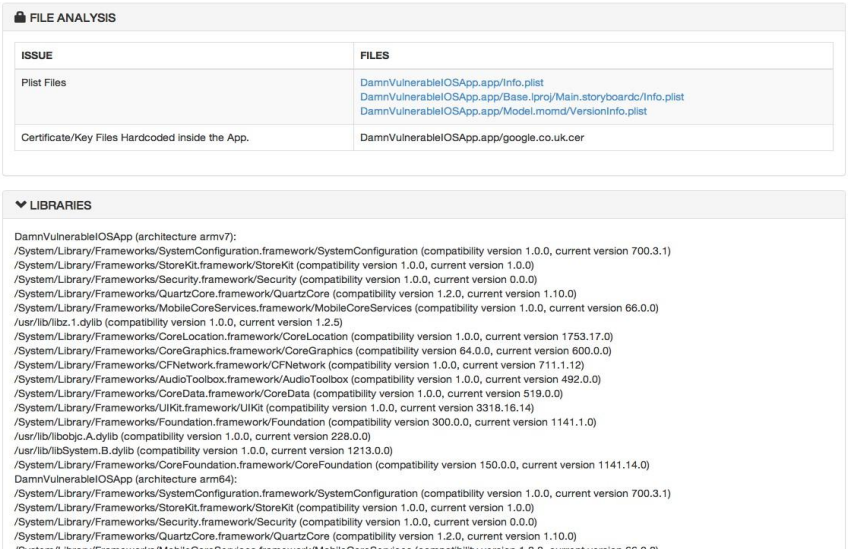
IPA BINARY ANALYSIS

Search:

PROTECTION	STATUS	SEVERITY	DESCRIPTION
ARC	True	info	The binary is compiled with Automatic Reference Counting (ARC) flag. ARC is a compiler feature that provides automatic memory management of Objective-C objects and is an exploit mitigation mechanism against memory corruption vulnerabilities.
CODE SIGNATURE	True	info	This binary has a code signature.
ENCRYPTED	False	warning	This binary is not encrypted.
NX	True	info	The binary has NX bit set. This marks a memory page non-executable making attacker injected shellcode non-executable.
PIE	True	info	The binary is build with -fPIC flag which enables Position independent code. This makes Return Oriented Programming (ROP) attacks much more difficult to execute reliably.
RESTRICTED	False	high	This binary does not have restricted segment that prevents dynamic loading of dylib for arbitrary code injection. Use compiler options -Wl,-sectcreate,__RESTRICT,__restrict,/dev/null to enable restricted segment.
RPATH	False	info	The binary does not have Runpath Search Path (@rpath) set.
STACK CANARY	True	info	This binary has a stack canary value added to the stack so that it will be overwritten by a stack buffer that overflows the return address. This allows detection of overflows by verifying the integrity of the canary before function return.
SYMBOLS STRIPPED	False	warning	Symbols are available. To strip debugging symbols, set Strip Debug Symbols During Copy to YES via the project's build settings.

Imagen 64 **Análisis de binario en MOBSF**

Análisis del fichero de la aplicación, librerías utilizadas y archivos disponibles: En este punto podremos acceder a los diferentes plist para examinarlos y ver también la librerías utilizadas en busca posibles vulnerabilidades.



FILE ANALYSIS

ISSUE	FILES
Plist Files	DamnVulnerableIOSApp.app/Info.plist DamnVulnerableIOSApp.app/Base.lproj/Main.storyboardc/Info.plist DamnVulnerableIOSApp.app/Model.momd/VersionInfo.plist
Certificate/Key Files Hardcoded inside the App.	DamnVulnerableIOSApp.app/google.co.uk.cer

LIBRARIES

DamnVulnerableIOSApp (architecture armv7):
/System/Library/Frameworks/SystemConfiguration.framework/SystemConfiguration (compatibility version 1.0.0, current version 700.3.1)
/System/Library/Frameworks/StoreKit.framework/StoreKit (compatibility version 1.0.0, current version 1.0.0)
/System/Library/Frameworks/Security.framework/Security (compatibility version 1.0.0, current version 0.0.0)
/System/Library/Frameworks/QuartzCore.framework/QuartzCore (compatibility version 1.2.0, current version 1.10.0)
/System/Library/Frameworks/MobileCoreServices.framework/MobileCoreServices (compatibility version 1.0.0, current version 66.0.0)
/usr/lib/libz.1.dylib (compatibility version 1.0.0, current version 1.2.5)
/System/Library/Frameworks/CoreLocation.framework/CoreLocation (compatibility version 1.0.0, current version 1753.17.0)
/System/Library/Frameworks/CoreGraphics.framework/CoreGraphics (compatibility version 64.0.0, current version 600.0.0)
/System/Library/Frameworks/CFNetwork.framework/CFNetwork (compatibility version 1.0.0, current version 711.1.12)
/System/Library/Frameworks/AudioToolbox.framework/AudioToolbox (compatibility version 1.0.0, current version 492.0.0)
/System/Library/Frameworks/CoreData.framework/CoreData (compatibility version 1.0.0, current version 519.0.0)
/System/Library/Frameworks/UIKit.framework/UIKit (compatibility version 1.0.0, current version 3318.16.14)
/System/Library/Frameworks/Foundation.framework/Foundation (compatibility version 300.0.0, current version 1141.1.0)
/usr/lib/libobjc.A.dylib (compatibility version 1.0.0, current version 228.0.0)
/usr/lib/libSystem.B.dylib (compatibility version 1.0.0, current version 1213.0.0)
/System/Library/Frameworks/CoreFoundation.framework/CoreFoundation (compatibility version 150.0.0, current version 1141.14.0)
DamnVulnerableIOSApp (architecture arm64):
/System/Library/Frameworks/SystemConfiguration.framework/SystemConfiguration (compatibility version 1.0.0, current version 700.3.1)
/System/Library/Frameworks/StoreKit.framework/StoreKit (compatibility version 1.0.0, current version 1.0.0)
/System/Library/Frameworks/Security.framework/Security (compatibility version 1.0.0, current version 0.0.0)
/System/Library/Frameworks/QuartzCore.framework/QuartzCore (compatibility version 1.2.0, current version 1.10.0)

Imagen 65 **Librerías IPA en MOBSF**

10.5.1.2. GHIDRA

Utilizaremos ahora este desensamblador para ver el código fuente de la aplicación. Una vez decompilado comenzaremos a realizar una serie de tareas:

Podemos empezar por comprobar las cadenas de texto presentes en el binario en el menú "Search"—> Search for Strings….

Las cadenas listadas pueden ser abrumadoras al principio, pero con algo de experiencia en la reversión de código Objective-C, aprenderás a filtrar y descartar las cadenas que no son realmente útiles o relevantes.

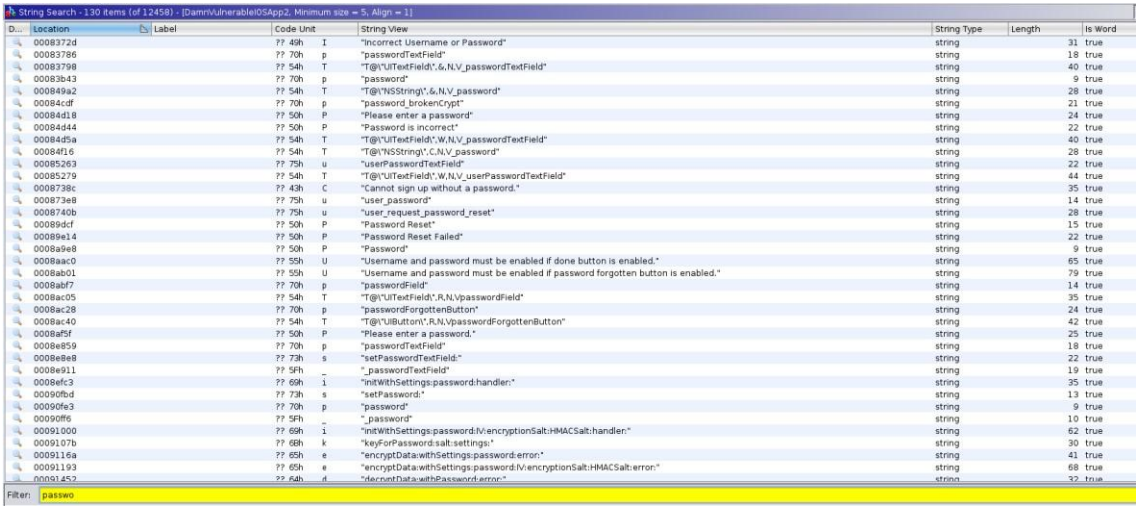


Imagen 66 **Cadenas de texto GHIDRA**

Utilizaremos el campo **filter** de la parte inferior para indicar filtros de cadena interesantes que me ayuden a encontrar urls, password incrustadas,nombres de bases de datos, símbolos, nombres de funciones,etc. Las utilizaremos cuando realicemos el análisis estático para comprobar si se está utilizando alguna función específica.

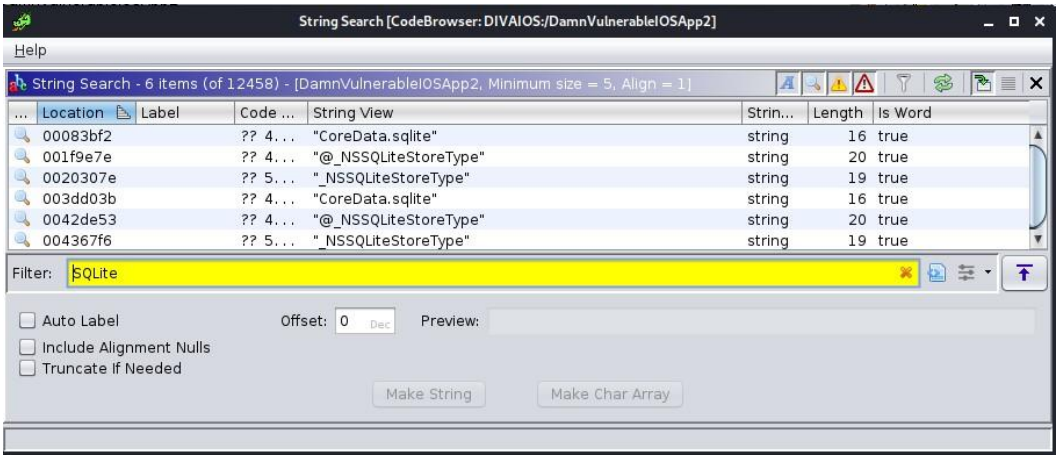


Imagen 67 **Búsqueda de cadenas de texto GHIDRA**

Campus Internacional CIBERSEGURIDAD
ENIIT INNOVA IT BUSINESS SCHOOL
UCAM UNIVERSIDAD CATÓLICA DE MURCIA

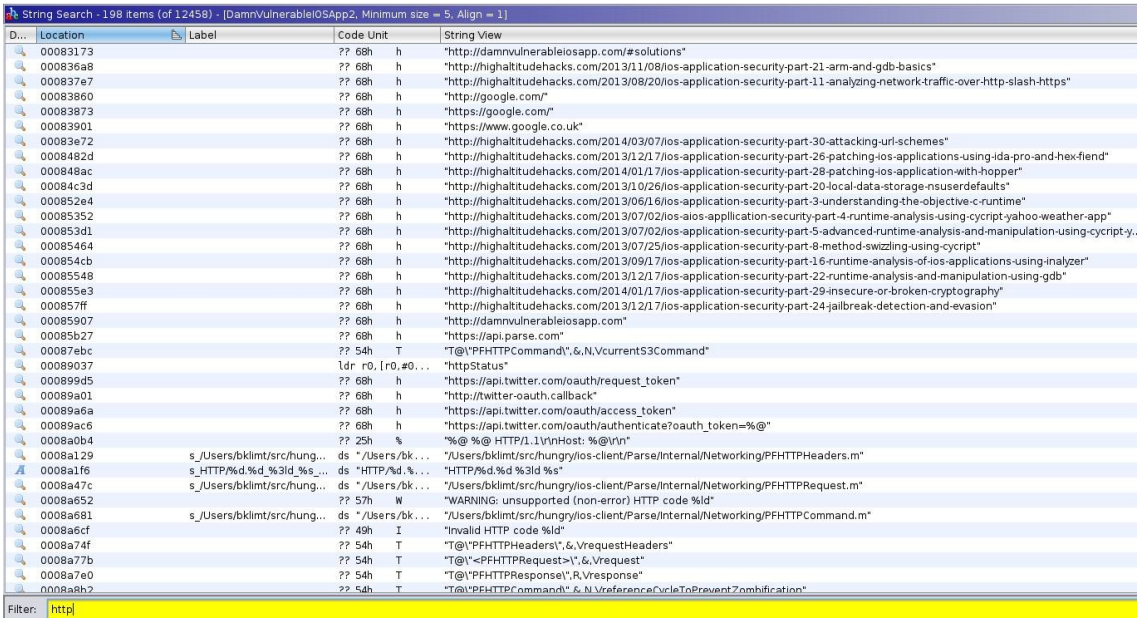


Imagen 68 **Búsqueda de cadena de texto "http" en GHIDRA**

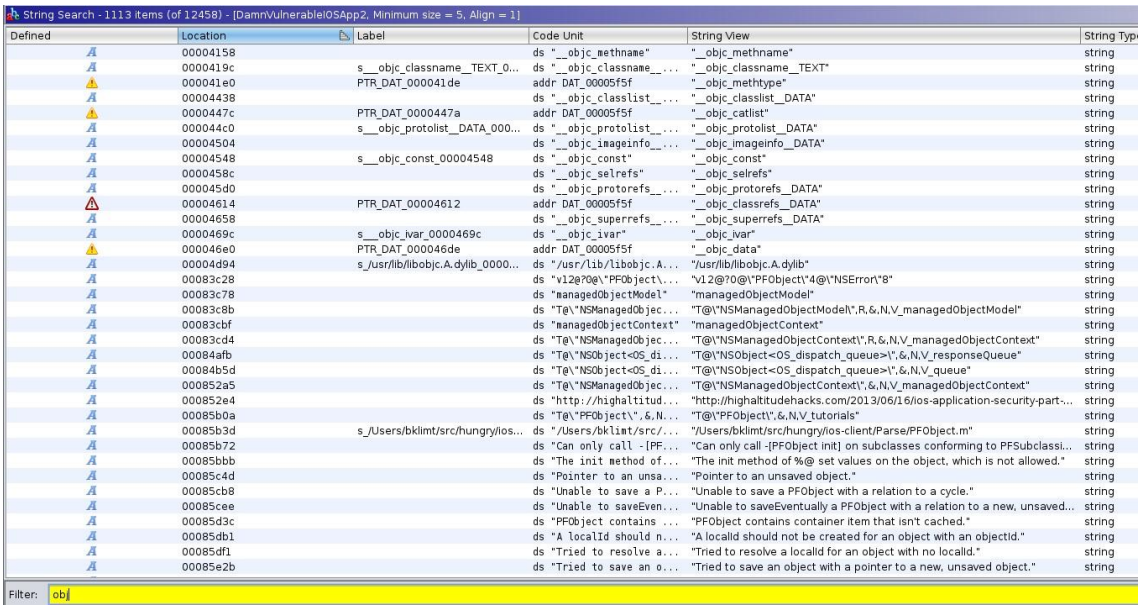


Imagen 69 **Búsqueda de cadena de texto "obj" en GHIDRA**

Como vemos disponemos del campo location, el cual podremos utilizar accediendo directamente a la dirección.

Para acceder a una dirección específica usaremos la tecla G que abrirá una ventana en la que introduciré la dirección deseada.

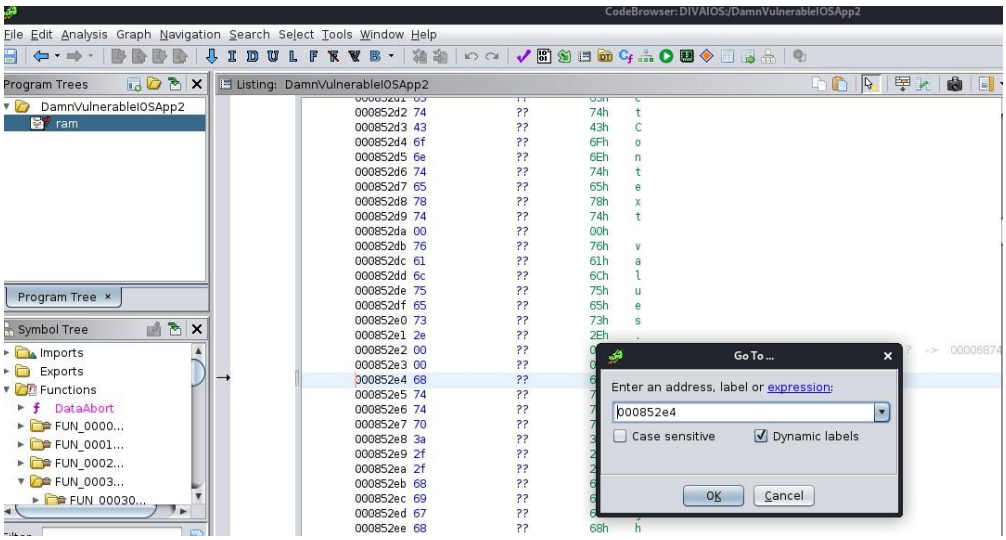


Imagen 70 **Acceso a dirección de memoria en GHIDRA**

Otra funcionalidad interesante es la de buscar referencias cruzadas en diferentes objetos, es decir que funciones utilizan otras llamadas. Para ello debemos situarnos sobre una función y con Click derecho de Raton vemos las opciones. Seleccionamos “Show references to”:

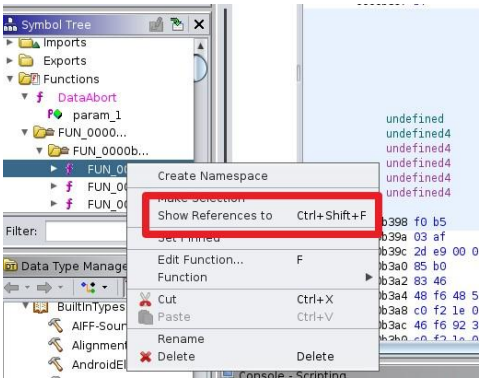


Imagen 71 **Búsqueda de referencias en GHIDRA**

Nos mostrará una ventana con las posibles funciones relacionadas:

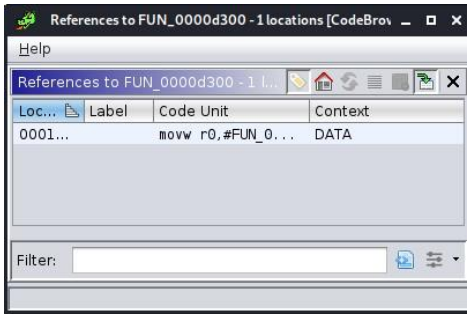


Imagen 72 **Referencias en GHIDRA**

Por supuesto podremos hacer doble click y navegar a la función, para observar en la parte central el código asm y a la derecha el código C en una transformación más legible.

11. ANÁLISIS DINÁMICO

11.1. ¿QUÉ ES?

Este tipo de análisis tiene como fin analizar y evaluar el comportamiento de una aplicación a través de su ejecución en tiempo real. El análisis dinámico suele utilizarse para comprobar si los mecanismos de seguridad ofrecen suficiente protección contra los tipos de ataque más frecuentes, como la divulgación de datos en tránsito, los problemas de autenticación y autorización y los errores de configuración del servidor.

Para ello podemos valernos de un abanico más o menos amplio de herramientas (en función del sistema operativo a analizar). En este caso vamos a enfocar el trabajo para realizar análisis de aplicaciones Androi, dadas las limitaciones expuestas para
IoS.

El análisis dinámico se lleva a cabo tanto en la capa de la plataforma móvil como en los servicios backend y las API, donde se pueden analizar los patrones de solicitud y respuesta de la aplicación móvil. En nuestro caso únicamente analizaremos la capa de la aplicación en la plataforma móvil dado que nos centraremos en el reversing de la aplicación y no en su backend.

Recordemos que los principales objetivos que se persiguen son:

- **Aprovechar los IoC e información obtenida en las comunicaciones de red**
- **Aprovechar información obtenida de símbolos**
- **Aprovechar información obtenida de strings**
- **Realizar volcado de clases**
- **Analizar métodos, funciones, argumentos…**
- **Ejecutar la aplicación en dispositivo**
- **Hacer un "hook" (engancharnos) al proceso**
- **Invocar métodos en tiempo de ejecución**

11.2. EJECUCIÓN DE LA APLICACIÓN

Realizaremos la ejecución sobre entornos de emulación con el objetivo de poder capturar la telemetría generada por la aplicación a lo largo de su ejecución. El Framework que utilizaremos incluye las herramientas de instrumentación necesarias para realizar un análisis dinámico completo.

11.3. ANÁLISIS DINÁMICO EN ANDROID

Para el análisis de aplicaciones Android vamos a utilizar una serie de programas y herramientas centralizadas en un Framework, que nos permitirán ejecutar la aplicación que estamos analizando en un dispositivo virtual controlado. De esta forma, podremos ver cómo se comporta la aplicación cuando interactuamos con ella a través de nuestro dispositivo virtual. Podremos ver cómo invocar partes de la aplicación que ha simple visto no están visibles o no podemos llegar a ellas por algún motivo (por ejemplo, con contraseña en la pantalla inicial). También vamos a ver como con algunas herramientas podemos llamar a partes de la aplicación que estamos analizando sin necesidad de interactuar con ella.

11.4. ¿DONDE REALIZAMOS EL ANÁLISIS?

Ya se vio anteriormente en el apartado dedicado al análisis estático de la aplicación cuales son los puntos interesantes sobre los que poner foco. Para el análisis dinámico, es decir, en el que se va a producir interacción por parte del usuario con la aplicación, pondremos especial antención a los "Intents".

También veremos que la aplicación puede realizar entre otras acciones comunicación con un backend, las cuales deben ser también analizadas permitiéndonos así entender mejor su funcionamiento.

En definitiva, debemos seguir las pistas del análisis estático para explorar todas las funcionalidades de la aplicación en los puntos de interés.

11.5. ANÁLISIS DINÁMICO EN LA PRÁCTICA

11.5.1 ADB E INSPECCIÓN DE LOG EN EJECUCIÓN (LOGCAT)

Anteriormente ya vimos que Adb nos sirve como interfaz de comunicación con el dispositivo móvil. En este caso veremos como inspeccionar el **registro de mensajes del sistema de Android. Logcat** es el comando de ADB que nos permite acceder a este registro. Podremos imprimir mensajes útiles para consultar el estado y posibles problemas que sucedan con la aplicación y en definitiva todo lo que pase por él durante la ejecución de la ejecución.

En este registro, las aplicaciones pueden incluir mensajes durante su uso, donde se incluyen la prioridad del registro, un identificador personalizado y el mensaje en cuestión. Existen seis tipos de mensajes, según su nivel de importancia, y que en Logcat aparecen generalmente abreviados con una letra:

V / Verbose: la menor prioridad, con más información de la necesaria
D / Debug: mensajes útiles para la depuración de aplicaciones
I / Info: mensajes de información
W / Warning: mensajes con advertencias
E / Error: errores de la aplicación
F / Fatal: errores graves

En resumen, el Logcat de Android es algo así como el cuaderno de bitácora de un móvil Android, donde las aplicaciones registran en segundo plano prácticamente todo lo que sucede en el móvil, desde mensajes internos de poca relevancia hasta errores que no llegan a notificarse al usuario.

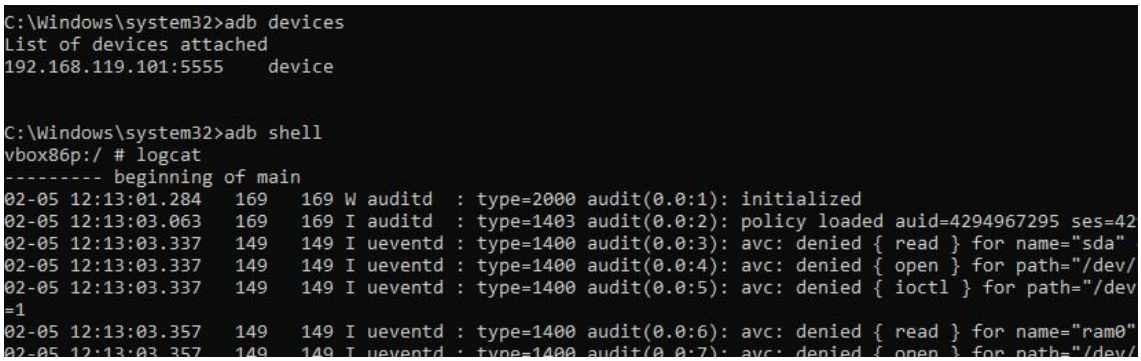


Imagen 73 **ADB conexión, Shell y logncat**

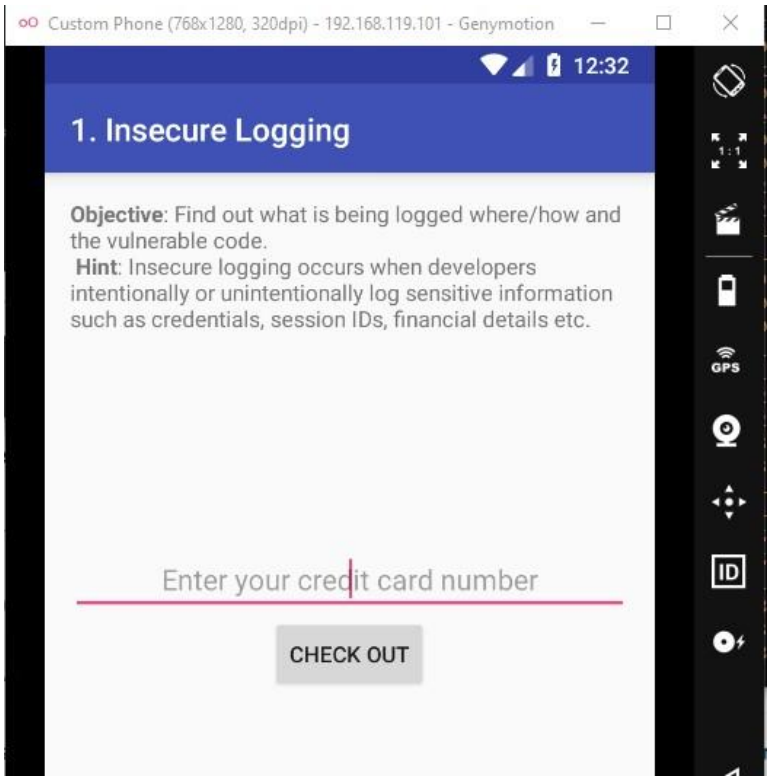


Imagen 74 **DIVA ejercicio 1**

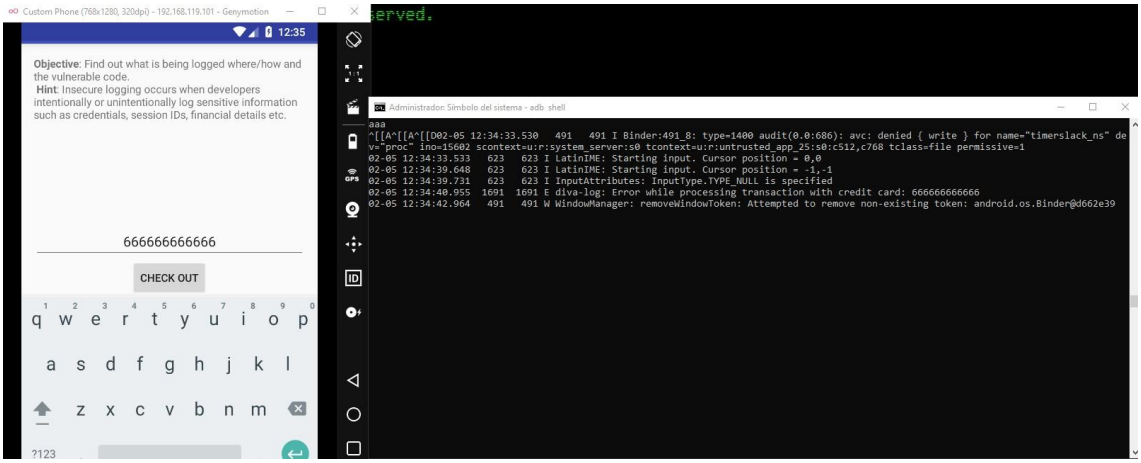


Imagen 75 **DIVA ejercicio 1 y logcat**



Imagen 76 **Análisis de código desensablado MOBSF**

11.5.2 MOBSF

Esta es otra de las herramientas que vamos a utilizar para llevar a cabo el análisis dinámico de una aplicación Android. Debido a su versatilidad, simplicidad y facilidad de uso, nos permite generar y emular un dispositivo virtual con Android, para poder "jugar" con las aplicaciones a analizar.

Una vez iniciado el dispositivo virtual, debemos arrancar el servicio web de MobSF. Para ello tendremos que situarnos en la consola, en el mismo directorio donde tenemos instalada la herramienta e iniciarla con el siguiente comando: "**run.bat**". Este comando levantará un servicio web al que podremos acceder desde nuestro navegador (por defecto en http://127.0.0.1:8000):

Una vez hemos seleccionado el fichero ".apk", la aplicación nos mostrará información relativa al mismo. Vamos a buscar la opción "Start Dynamic analysis" y ejecutarla:

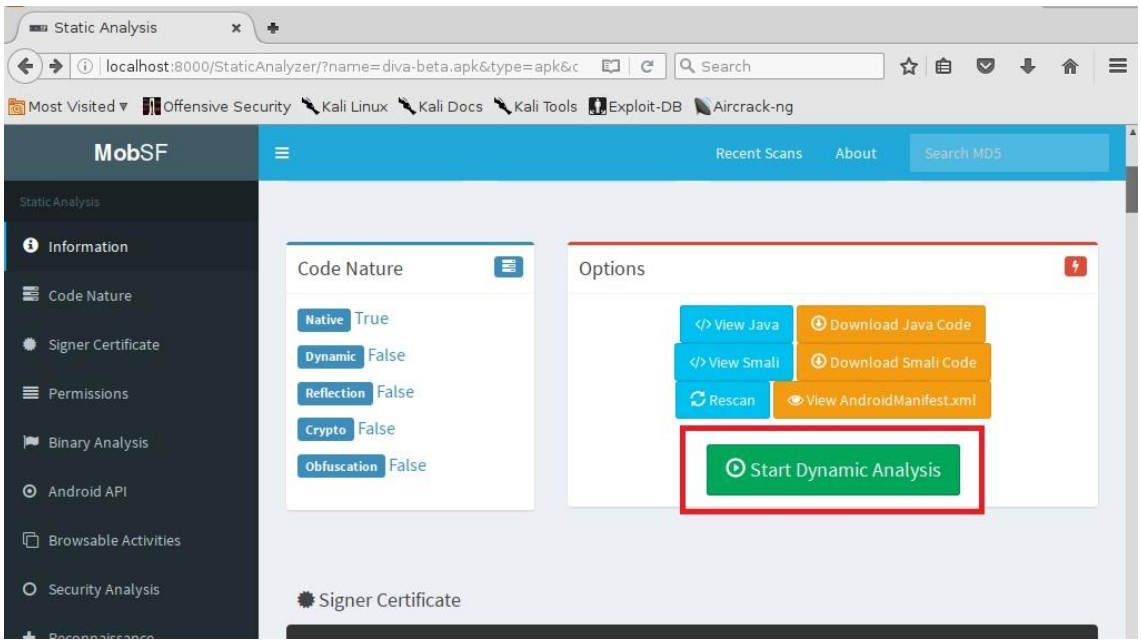


Imagen 77 **Análisis dinámico MOBSF**

En la siguiente pantalla tendremos que indicarle a la aplicación que nos cree el entorno necesario para iniciar el análisis dinámico. Para ello, pulsamos el botón "Create Environment":

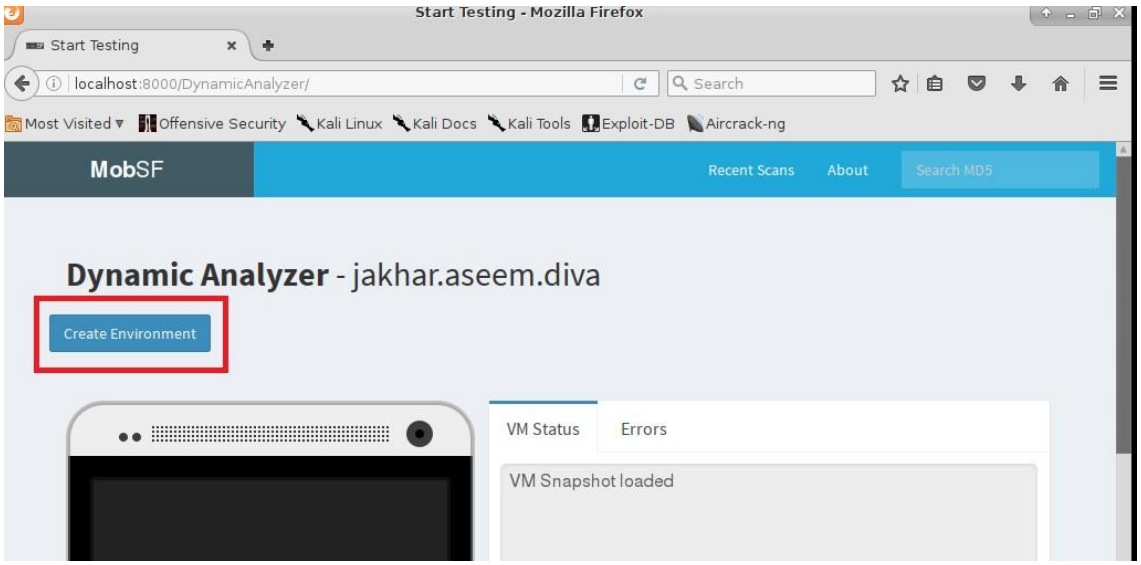


Imagen 78 **Análisis dinámico MOBSF, creación de entorno**

Una vez que la herramienta ha generado el entorno, veremos cómo nos muestra una serie de opciones que no teníamos disponibles antes y que nos van a ayudar a realizar en análisis de la aplicación:

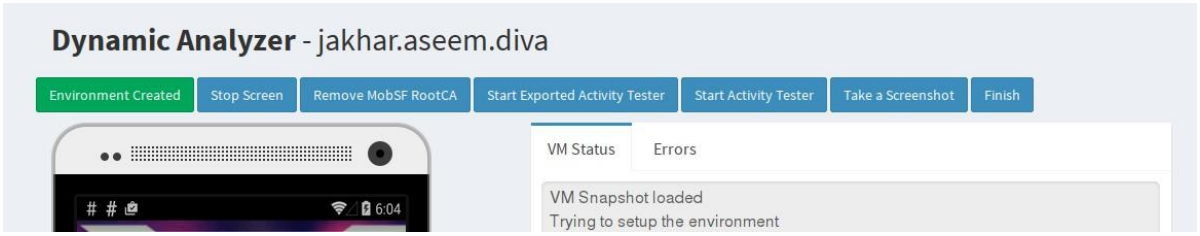


Imagen 79 **Análisis dinámico MOBSF**

También tenemos disponible, en la parte de la derecha, información sobre el estado de la máquina virtual de Android y los posibles errores que se vayan generando:

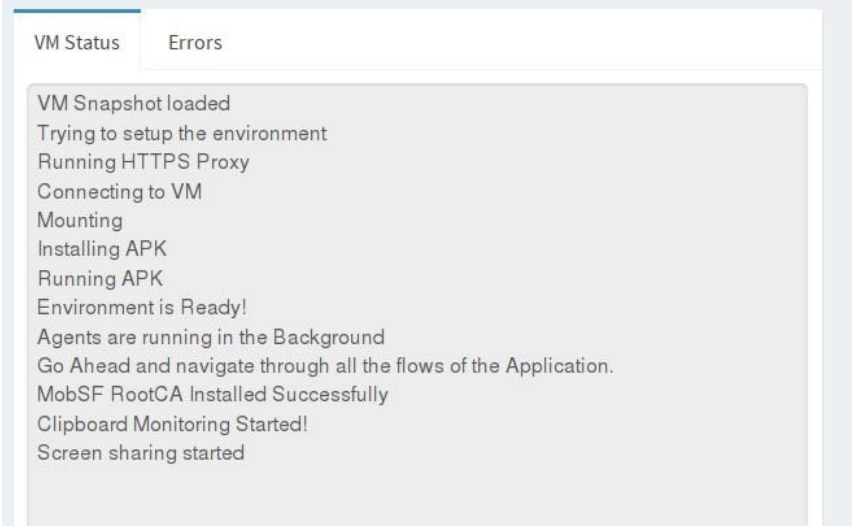


Imagen 80 **Análisis dinámico MOBSF, Logs aplicación**

Desde esta pantalla podemos replicar la pantalla de nuestros dispositivo virtual ("Show Screen"), remover el certificado que la herramienta instala en el dispositivo ("Remove MobSF RootCA"), iniciar los diferentes activities de la aplicación de forma automática ("Start Exported Activity Tester" y "Start Activity Test"), tomar un screenshot de la pantalla ("Take a Screenshot") y finalizar en análisis dinámico
("Finish").

En la parte inferior podemos lanzar comandos "adb" para comunicarnos con la aplicación desde el entorno virtual:

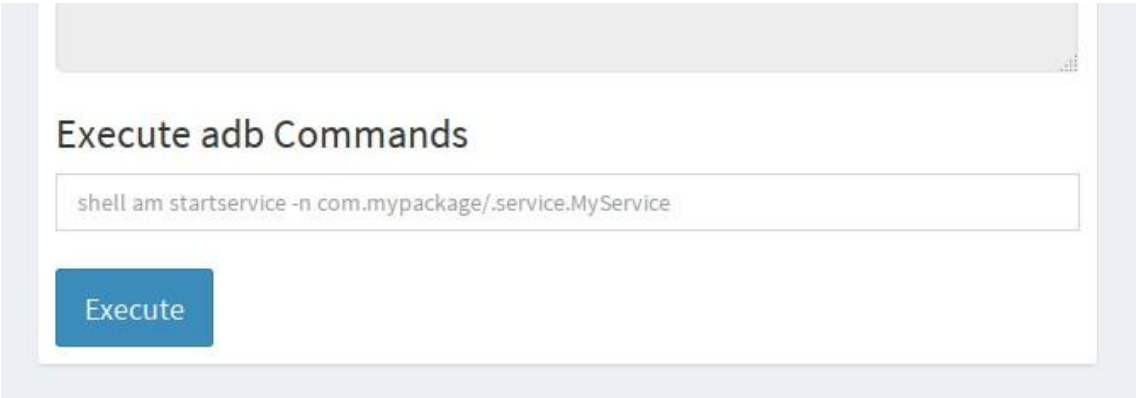


Imagen 81 **Análisis dinámico MOBSF, ejecución de comandos ADB**

Hay que destacar que toda interacción que hagamos con la aplicación dentro del entorno virtual quedará registrado por la herramienta y posteriormente podremos analizar los datos recopilados.

Durante este proceso tenemos que interactuar con la aplicación con el fin de "forzarla" a utilizar los diferentes elementos que la componen y así facilitar al programa la recopilación de datos.

Una vez finalizado el análisis la herramienta nos mostrará una serie de opciones relativas a la información que ha recopilado sobre la aplicación. En el listado de opciones de la parte izquierda podremos ir viendo uno a uno los diferentes tipos de datos que hay ido recopilando: llamadas a la api, información sobre el dispositivo, ficheros que utiliza la aplicación, bases de datos que haya podido utilizar…

Bases de datos SQLite:

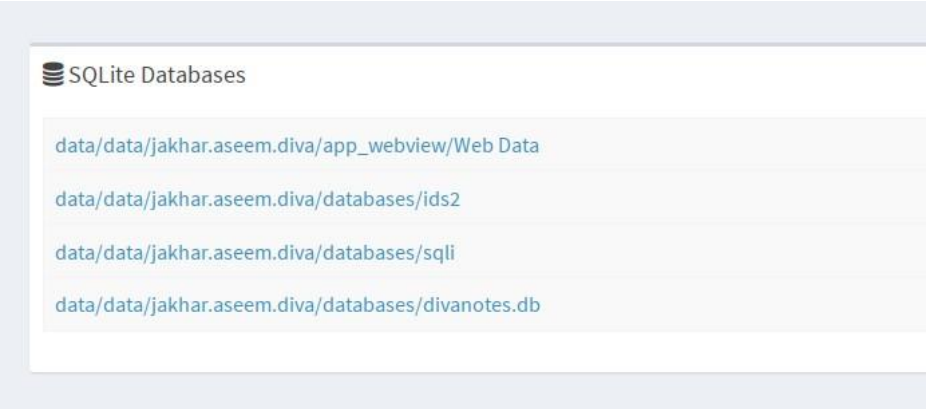


Imagen 82 **Análisis dinámico MOBSF, Bases de datos**

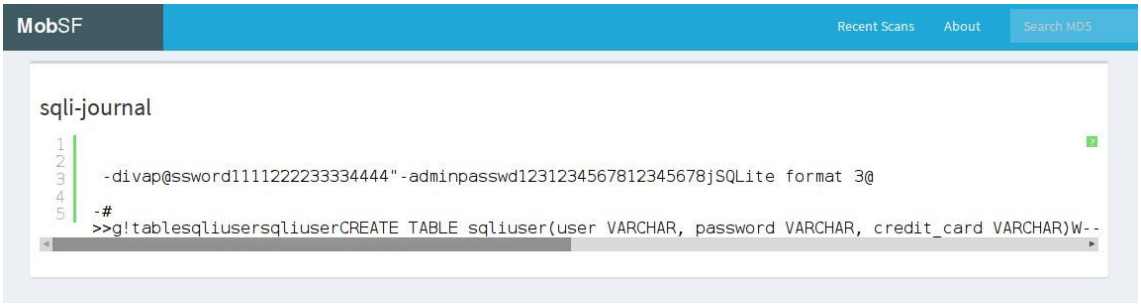


Imagen 83 **Análisis dinámico MOBSF, BD sqli-journal**

Información sobre las llamadas a los activities:



Imagen 84 **Análisis dinámico MOBSF, Actividades**

Listado de los activities que tiene la aplicación:



Imagen 85 **Análisis dinámico MOBSF, prueba de Actividades**

El uso de diferentes herramientas a la hora de analizar aplicaciones Android nos da diferentes puntos de vista a la hora de recolectar datos. Todas ellas realizan acciones contra la aplicación (cada una en su entorno) y es posible que una no detecte o no sea capaz de interpretar ciertos resultados que la otra sí ha podido, con lo que es recomendable el uso de varias herramientas diseñadas con el mismo fin para tener una visión global del análisis de la aplicación.

12. EJERCICIOS PRÁCTICOS

En llos siguientes enlaces puedes acceder a diferentes ejercicios de análisis de aplicaciones. Contienen tanto los binarios para su análisis como una guía de su realización.

- Enlace 1
- Enlace 2
- Enlace 3
- Enlace 4

13. REFERENCIAS

- https://ragingrock.com/AndroidAppRE/app_fundamentals.html
- https://www.slideshare.net/TomKeetch/steelcon-2015-reverseengineeringobfuscated-android-applications
- https://cybercamp.es/sites/default/files/contenidos/material/cybercamp2016 - pentesting_de_aplicaciones_ios-miguel_angel_arroyo.pdf
- https://cybercamp.es/sites/default/files/contenidos/material/cybercamp2016 - pentesting_de_aplicaciones_ios-miguel_angel_arroyo.pdf
- https://docs.microsoft.com/es-es/windows/wsl/install-win10#step-4--download-the-linux-kernel-update-package
- https://developer.android.com/guide/components/broadcasts#receivingbroadcasts
- http://damnvulnerableiosapp.com/?paiddownloads_id=16
- https://www.youtube.com/watch?v=xDyLy0zLV7s